PRAWNS: Threshold Hash-Based Signatures from Threshold PRFs

Dan Boneh[1], Benedikt Bünz[2], Justin D.[3], Yavor Litchev[1], and Kamilla Nazirkhanova[1]

[1]Stanford University
[2]NYU
[3]Localhost

September 21, 2026

Abstract

We propose an efficient threshold mechanism that is well suited for certain hash-based signatures. Our proposal has a number of compelling features:

- The final signature assembled from a quorum of signature shares has the same format as a non-threshold hash-based signature. In particular, the verifier is unaware that the signer is thresholdized. Moreover, we can add and remove parties and change the threshold without changing the public key.
- Our framework can be used to thresholdize one-time signature schemes, such as Winternitz, and a few-time schemes, such as XMSS, HORS, FORS, and PORS+FP.
- The public key and the signatures reveal nothing about the number of parties n or the threshold t.
- An instantiation of our framework scales well to a large number of parties n and a large threshold t.

One limitation of the framework is that it does not support an efficient distributed key generation protocol (DKG). Another limitation is that the framework does not directly apply to SLH-DSA. It is primarily designed for the Winternitz, XMSS, and FORS schemes.

Contents

1 Introduction **3**

2 Context-aware threshold PRFs **5**

3 The threshold signature scheme **7**
3.1 Threshold Winternitz-OTS using a threshold PRF 8
3.2 Security of threshold one-time signatures 10
3.3 Security of our construction 11
3.4 Applications beyond WOTS 13

4 A lattice-based context-aware threshold PRF **15**
4.1 Security from LWE 17

5 A hash-based context-aware threshold PRF **21**
5.1 Making the scheme context aware 22

6 Extensions and open problems **27**

References **28**

A Proof of Lemma 4.2 **32**

1 Introduction

The quantum risk to blockchains is generating interest in post-quantum signatures. While NIST chose two lattice-based signature schemes [39, 22] that are believed to be post-quantum secure, some blockchains are reluctant to introduce new complexity assumptions in the core architecture. As a result, there is renewed interest in hash-based signatures. Some recent proposals include NIST's SLH-DSA standard [40] based on Sphincs$^+$ [6], the Bitcoin Shrincs scheme [15], and Ethereum's LeanSig [21].

One difficulty with hash-based signatures is their lack of support for threshold signing. A simple solution is to use the trivial threshold signature (also called multisig) where a t-of-n signature is built by concatenating t non-threshold signatures from t of n parties. However, the signature length and verification time is linear in the threshold t, which can be difficult for a large t. Moreover, the signature reveals the threshold t which is undesirable in some applications. One way around these issues is to compress the t signatures into a short proof using a hash-based SNARK proof system. However, this requires the verifier to verify a SNARK proof, which adds significant software complexity to the blockchain system. One can also use generic MPC techniques to thresholdize hash-based signatures, but experiments [2, 16] suggest that this can generate significant network traffic even for a small threshold t.

Haystack [31, 35] is a combinatorial solution to thresholdizing one-time signatures. The construction only works when $\binom{n}{t}$ is small and every signature reveals both n and t. Moreover, the threshold signature format differs from that of a standard hash-based signature.

Our results. We propose using a context-aware threshold Pseudorandom Function (PRF) to construct a threshold hash-based signature scheme. Our scheme, called PRAWNS[1], outputs a regular (non-threshold) hash-based signature. The standard notion of a threshold PRF [34, 37, 36, 38] is insufficient for our goals. Instead, in Section 2 we strengthen threshold PRFs to also include an evaluation context, which limits how evaluation shares can be combined to obtain the final PRF value. We call such a PRF a *context-aware threshold PRF*.

In Section 3 we describe our threshold signature as it applies to the Winternitz one-time signature scheme. The same approach applies to other hash-based schemes [32], such as XMSS [12], HORS [43], PORS [3, 1], and FORS [6]. We discuss these schemes in Section 3.4. In all these schemes the verifier is unaware that the signer is thresholdized. Variants of XMSS and FORS are proposed as standalone signature schemes for Bitcoin and Ethereum, for example, in the Shrincs proposal [15]. We also explain why our approach does not easily apply to SLH-DSA [40], although it can apply to a scaled down version of SLH-DSA using a technique called hypertree pruning [14].

In Sections 4 and 5 we construct two context-aware threshold PRFs. One of them is purely hash-based, but is only efficient when $\binom{n}{t}$ is small. Our main construction supports larger values of n and t, and builds on a lattice PRF whose security is based on the standard LWE lattice assumption [42]. We stress, however, that the lattice PRF is only used by the threshold signer. The verifier is purely hash based, and is unaware that the signer is thresholdized or that it uses a lattice PRF. Specifically, in the blockchain setting, blockchain nodes are unaware of lattices, and the blockchain architecture does not rely on lattice assumptions. It is entirely up to the signer to decide if it wants to thresholdize its signing key, and how to do so.

[1] The name PRAWNS stands for "PRF Assisted Winternitz for n Signers."

Defining security for threshold one-time signatures. Since our main construction thresholdizes the Winternitz one-time signature (OTS) scheme, we must address some subtleties in defining threshold OTS security. A non-threshold OTS scheme is secure as long as the signer never signs more than one message with respect to a particular public key. This leads to an immediate problem in the threshold setting. Imagine one quorum of t parties is asked to sign one message m and another disjoint quorum of t parties is asked to sign some other message m'. If the signers do not coordinate with each other then both t-size quorums will issue the respective signatures. These two signatures under the same public key can break the security of the OTS and enable the adversary to forge the signatures on some other message $m'' \neq m, m'$.

We do not want to require coordination among the n signing parties. The only guarantee we can expect is that an honest signer will not issue a signature share on two messages with respect to a single public key. As a result, if some $f < t$ parties are corrupt, we can ensure security only when the threshold t satisfies $t > (n+f)/2$. This threshold ensures that if an attacker requests a quorum of size t to sign one message and another quorum of size t to sign another message, then some honest party must sign both messages with respect to the same public key. That honest party will refuse and the attacker will fail to obtain two signed messages under a single public key. Hence, we can only ensure security for a sufficiently large threshold. For example, if at most $f \leq n/3$ parties are corrupt, then the threshold for constructing a signature must satisfy $t > (2/3)n$. This is inherent for the security of any threshold OTS scheme where the signers do not communicate with each other, and when two valid signatures under a single public key can compromise security. We give the complete security definition in Section 3.2. We note that the inherent gap between f and t is related to the notion of *dual threshold* in threshold signatures [13]. For a "few" time signature scheme, we can relax the constraint on t, as explained next.

Few-time signatures. While we focus on one-time signatures, all of our constructions apply equally well to a *stateless* w-time hash-based signature scheme (such as HORS [43] and FORS [32]), where the scheme remains secure as long as no more than $w \geq 1$ signatures are issued with respect to a single public key. For a threshold w-time signature, we know that an honest signer will sign at most w messages for a single public key. This means that the adversary can obtain at most $w(n-f)$ signature shares from the honest parties. If the adversary corrupts f parties, it needs $t-f$ honest shares to obtain a signature. Therefore, the number of signatures it can obtain under a single public key is at most $w(n-f)/(t-f)$. For example, when $t = (2/3)n$ and $f \leq n/3$ the adversary can obtain at most $2w$ signatures. Hence, for $t = (2/3)n$ and $f \leq n/3$, if we start from a signature scheme that is $2w$-time secure, we obtain a threshold signature scheme that is secure as long as every honest party signs at most w messages, without requiring any coordination among the n signing parties. Similar arguments are used in some consensus protocols [45].

We can do better when w is small, say $w = 2, 3, 4$, and the threshold t is large, namely $t > (wn+f)/(w+1)$. This high threshold ensures that if the adversary attempts to obtain $w+1$ signatures, then some honest party must sign $w+1$ messages. The honest party will refuse to sign that many times and the adversary will be unable to obtain $w+1$ signatures for a single public key. For example, when $f = n/3$ we need $t > \frac{3w+1}{3w+3} \cdot n$. For $w = 2$ this gives $t > 0.78n$. When using such a high threshold t, if the scheme is w-time secure, we obtain a threshold signature scheme that is w-time secure, without requiring any coordination among the n signing parties. This is better than setting the parameters so that the scheme is $2w$-time secure, as needed in the previous paragraph when $t = 2n/3$.

We aim to design threshold signature schemes where every signer can operate on its own, without communicating with the other signers. If signers can coordinate with one another to make sure that a one-time public key is only used to sign one message, then the restrictions in the previous three paragraphs are not needed.

Notation. In what follows we use $[n]$ to denote the set $\{1, 2, \ldots, n\}$ and use $[0, n)$ to denote $\{0, 1, \ldots, n-1\}$. We use $\binom{[n]}{t}$ to denote the set of all t-size subsets of $[n]$. Thus, the set $\binom{[n]}{t}$ contains $\binom{n}{t}$ elements, where each element is a t-size subset of $[n]$. As usual, we use $x \leftarrow y$ to denote assignment, and $x \leftarrow\!\!\$\, \mathcal{X}$ to denote a uniform random variable x over $\mathcal{X}$. We use $\mathrm{Funs}[\mathcal{X}, \mathcal{Y}]$ to denote the set of all functions from $\mathcal{X}$ to $\mathcal{Y}$. Throughout the paper, the sets $\mathcal{K}, \mathcal{S}, \mathcal{X}, \mathcal{Y}, \mathcal{Z}$ should be understood to be ensembles of finite sets, for example $\mathcal{X} = \{\mathcal{X}_\lambda\}_{\lambda=1}^{\infty}$. Similarly, a function $F : \mathcal{K} \times \mathcal{X} \to \mathcal{Y}$ should be understood to be an ensemble of functions $\{F_\lambda : \mathcal{K}_\lambda \times \mathcal{X}_\lambda \to \mathcal{Y}_\lambda\}_{\lambda=1}^{\infty}$. This lets us refer to a PPT adversary $\mathcal{A}$ that runs in polynomial time in the security parameter λ, where λ is used to select the appropriate sets and functions from these ensembles. All our algorithms take the security parameter 1^λ as their first input, however for brevity we will occasionally omit that input.

2 Context-aware threshold PRFs

Our core building block is a threshold pseudorandom function, or threshold PRF, also known as distributed PRF. They were introduced by Micali and Sidney [34] and have been extensively studied [19, 20, 37, 36, 38]. More recent work has constructed efficient threshold PRFs from lattice assumptions [9, 4, 33]. We discuss lattice-based threshold PRFs in more detail in Section 4.

The standard notion of a threshold PRF is insufficient for our application. In a standard threshold PRF, an evaluation share is associated only with the PRF input x. For our threshold signature construction we need a stronger property. An evaluation share must also be tied to an *evaluation context* ec. This evaluation context ec ensures that evaluation shares for x obtained under different contexts cannot be combined to learn the value of $F(\mathsf{msk}, x)$. The notion of a context was recently introduced in the setting of threshold decryption [7]. Here we adapt this idea to a threshold PRF.

We next define the syntax and security of a context-aware threshold PRF. We modify the evaluation algorithm Eval and let it take an evaluation context as an additional input.

Definition 2.1. A **context-aware threshold PRF** is a triple of PPT algorithms as follows:

- $\mathsf{KeyGen}(1^\lambda, n, t) \to (\mathsf{msk}, \mathsf{sk}_1, \ldots, \mathsf{sk}_n)$. Outputs a master evaluation key msk along with n evaluation keys in $\mathcal{K}$ for a PRF $F : \mathcal{K} \times \mathcal{X} \to \mathcal{Y}$.
- $\mathsf{Eval}(1^\lambda, \mathsf{sk}_i, x \in \mathcal{X}, \mathsf{ec}) \to y_i$. Takes as input an evaluation point $x \in \mathcal{X}$ and an evaluation context ec, and outputs a PRF evaluation share y_i.
- $\mathsf{Combine}(1^\lambda, \mathcal{J} \in \binom{[n]}{t},\ x,\ \{y_i\}_{i \in \mathcal{J}}) \to y$. Takes as input t evaluation shares $\{y_i\}_{i \in \mathcal{J}}$ and outputs the PRF evaluation $y = F(\mathsf{msk}, x)$, if all the shares $\{y_i\}_{i \in \mathcal{J}}$ were generated with respect to the same evaluation context ec.

The threshold PRF must satisfy the following correctness property: for every $\mathsf{msk}, \mathsf{sk}_1, \ldots, \mathsf{sk}_n$ in the support of $\mathsf{KeyGen}(1^\lambda, n, t)$, every quorum $\mathcal{J} \subseteq [n]$ of size t, every $x \in \mathcal{X}$, and every evaluation

context ec, we have that

$$\Pr\left[F(\mathsf{msk}, x) = \mathsf{Combine}\big(\mathcal{J}, x, \{\mathsf{Eval}(\mathsf{sk}_i, x, \mathsf{ec})\}_{i\in\mathcal{J}}\big)\right] = 1$$

Security. The security game is presented in Figure 1. Informally, security provides the following guarantee: suppose there are at most $f < t$ corrupt parties, and the adversary gets to request an evaluation share from any party of its choice at any $x \in \mathcal{X}$ of its choice. Then the adversary cannot distinguish the value of $F(\mathsf{msk}, x)$ from random, unless it has requested $t - f$ evaluation shares for x under a common evaluation context ec.

Definition 2.2. A context-aware threshold PRF $\mathcal{F} = (\mathsf{KeyGen}, \mathsf{Eval}, \mathsf{Combine})$ is **adaptively secure** if for all PPT adversaries $\mathcal{A}$ the following advantage is negligible

$$\mathsf{Adv}^{\mathrm{tprf}}_{\mathcal{A},\mathcal{F}}(\lambda) := \Big|\Pr\big[\mathbf{G}^{\mathrm{tprf}}_{\mathcal{A},\mathcal{F}}(\lambda, 0) = 1\big] - \Pr\big[\mathbf{G}^{\mathrm{tprf}}_{\mathcal{A},\mathcal{F}}(\lambda, 1) = 1\big]\Big|.$$

If $\mathcal{A}$ must issue all its queries to the oracle $\mathsf{OCorrupt}(\cdot)$ at the beginning of the game then we say that $\mathcal{F}$ is **statically secure** and we denote the corresponding advantage by $\mathsf{Adv}^{\mathrm{stprf}}_{\mathcal{A},\mathcal{F}}(\lambda)$.

Robustness. One can also add a robustness property that prevents a malicious party from disrupting the evaluation process. This property is often achieved using a NIZK proof system to ensure that every party correctly computes its evaluation share. We discuss this further in Section 4.

Constructing a threshold PRF. We construct two context-aware threshold PRFs in Sections 4 and 5.

The Game $\mathbf{G}^{\mathrm{tprf}}_{\mathcal{A},\mathcal{F}}\big(\lambda,\ b \in \{0,1\}\big)$

$C \leftarrow \emptyset$ // the set of challenge points
$Q \leftarrow \emptyset$ // an associative array of queried points
$D \leftarrow \emptyset$ // the set of corrupt parties
$f \leftarrow\$ \mathrm{Funs}[\mathcal{X}, \mathcal{Y}]$ // sample a random $f : \mathcal{X} \to \mathcal{Y}$
$(n, t, \mathsf{state}) \leftarrow\$ \mathcal{A}(1^\lambda)$
$(\mathsf{msk}, \mathsf{sk}_1, \ldots, \mathsf{sk}_n) \leftarrow\$ \mathsf{KeyGen}(1^\lambda, n, t)$
$b' \leftarrow\$ \mathcal{A}^{\mathsf{OCorrupt},\mathsf{OEval},\mathsf{OChallenge}}(1^\lambda, \mathsf{state})$
return $b' \in \{0,1\}$

Oracle $\mathsf{OCorrupt}(i)$

if $\exists\ x, \mathsf{ec}$ s.t. $\big|D \cup Q[x, \mathsf{ec}] \cup \{i\}\big| \geq t$
  and $x \in C$: **return** $\perp$
$D \leftarrow D \cup \{i\}$
return sk_i

Oracle $\mathsf{OChallenge}(x)$

if $\exists \mathsf{ec} : \big|D \cup Q[x, \mathsf{ec}]\big| \geq t$:
  return $\perp$
$y \leftarrow f(x)$
if $b = 0 : y \leftarrow F(\mathsf{msk}, x)$
$C \leftarrow C \cup \{x\}$
return y

Oracle $\mathsf{OEval}(i, x, \mathsf{ec})$

if $\big|D \cup Q[x, \mathsf{ec}] \cup \{i\}\big| \geq t$
  and $x \in C$: **return** $\perp$
$Q[x, \mathsf{ec}] \leftarrow Q[x, \mathsf{ec}] \cup \{i\}$
return $\mathsf{Eval}(1^\lambda, \mathsf{sk}_i, x, \mathsf{ec})$

Figure 1: The security game for a context-aware threshold PRF $F : \mathcal{K} \times \mathcal{X} \to \mathcal{Y}$ with algorithms $(\mathsf{KeyGen}, \mathsf{Eval}, \mathsf{Combine})$.

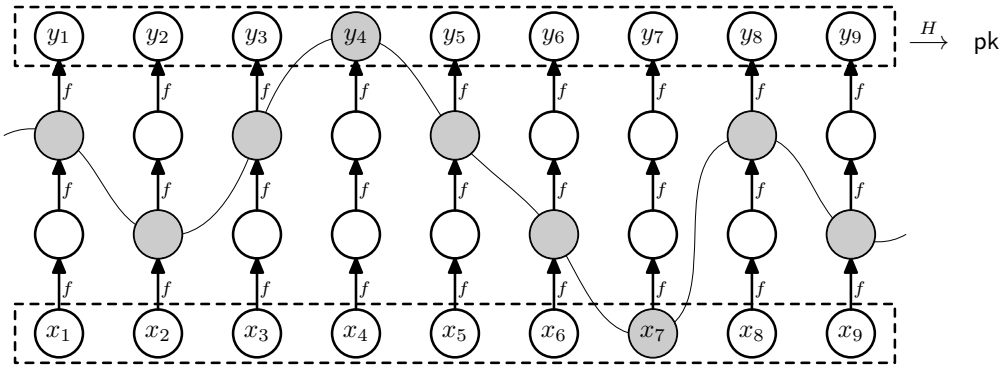


Figure 2: The rectangle of values generated during WOTS key generation with $\ell = 9$ and $d = 4$. The shaded nodes represent a signature on a particular message m where $P(m) = (2, 1, 2, 3, 2, 1, 0, 2, 1)$.

3 The threshold signature scheme

We now describe the threshold signature mechanism as it applies to Winternitz. First, we briefly review the Winternitz one-time signature scheme (WOTS), and refer to [10, p. 605] or [32] for a detailed description. To simplify the exposition we describe the scheme using a *non-keyed* hash function $f : \mathcal{Y} \to \mathcal{Y}$. We discuss the relation to the exact variant of Winternitz used in Sphincs$^+$ [6] in Remark 1 below.

The WOTS scheme is parameterized by two integers ℓ and d and three functions:

- a collision-resistant hash function $P : \mathcal{M} \to [0, d)^\ell$, (see Remark 4 for how to relax P so that we only rely on target collision resistance)
- a hash function $f : \mathcal{Y} \to \mathcal{Y}$ where $\mathcal{Y} = \{0, 1\}^\kappa$ for some κ,
- a target collision resistant (TCR) hash function $H : \mathcal{K} \times \mathcal{Y}^\ell \to \mathcal{Z}$.

During key generation the key generator first samples random keys $(x_1, \ldots, x_\ell)$ in $\mathcal{Y}^\ell$ and a key k in $\mathcal{K}$. The secret key is $\mathsf{sk} := (x_1, \ldots, x_\ell, k)$. Next, the key generator uses the function f to build a hash-chain of length d starting with each x_i, as shown in Figure 2. The key generator obtains $(y_1, \ldots, y_\ell) \in \mathcal{Y}^\ell$ as the top of the ℓ chains. Finally, it sets the public key to be

$$\mathsf{pk} \leftarrow \big(H(k, y_1, \ldots, y_\ell),\ k\big) \in \mathcal{Z} \times \mathcal{K}.$$

Figure 2 shows the rectangle of values generated during key generation for $\ell = 9$ and $d = 4$.

A signature on a message $m \in \mathcal{M}$ contains exactly ℓ values in $\mathcal{Y}$. To sign m the signer computes $P(m) \in [0, d)^\ell$ and reveals the relevant values in the rectangle. The shaded circles in Figure 2 represent the signature on a message m where $P(m) = (2, 1, 2, 3, 2, 1, 0, 2, 1)$. This $P(m)$ describes a cut through the rectangle illustrated by the thin line in the figure. Verifying a signature on m with respect to pk is done by computing $P(m)$ and checking that the values provided in the signature are consistent with the public key. For this signature scheme to be one-time secure, the function P must satisfy the following cover free property [10, p. 605].

Definition 3.1. For a function $P : \mathcal{M} \to [0, d)^\ell$ we say that m covers m' if $P(m)[i] \le P(m')[i]$ for all $i \in [\ell]$, and then write $P(m) \le P(m')$. We say that a function $P : \mathcal{M} \to [0, d)^\ell$ is **cover free** if m does not cover m' for all $m \ne m' \in \mathcal{M}$.

Remark 1 (Relation to Winternitz in Sphincs$^+$). The hash function in Figure 2 has domain $\mathcal{Y}$. Winternitz, as used in Sphincs$^+$ [6], makes use of a keyed tweakable hash function which is a hash function with an expanded domain $\mathcal{K} \times \mathcal{T} \times \mathcal{Y}$, where $\mathcal{K}$ is a key space and $\mathcal{T}$ is a tweak space. The random key and the tweak let Sphincs$^+$ obtain a tight security reduction to the underlying assumptions using ideas from [28, 29]. In our proof of security (Theorem 3.1) the security reduction introduces a factor of ℓd when reducing to the underlying assumption, but this lets us prove security based on a simple second preimage resistance assumption on f (Definition 3.3). The analysis in Sphincs$^+$ eliminates this factor of ℓd to obtain a tight reduction, but requires considerably stronger assumptions on f. Our threshold constructions work equally well with a keyed tweakable hash function as in Sphincs$^+$, but for simplicity, here we will use a hash function f without keys and tweaks.

3.1 Threshold Winternitz-OTS using a threshold PRF

Let $(\mathsf{KeyGen}, \mathsf{Eval}, \mathsf{Combine})$ be a threshold PRF as defined in Section 2, with domain $\mathcal{X} := [\ell] \times [0, d)$ and range $\mathcal{Y}$. Recall that $F(\mathsf{msk}, x)$ denotes the output of the PRF on input $x \in \mathcal{X}$ and the (non-threshold) key msk.

Our threshold scheme is a tuple of four algorithms $(\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Combine}, \mathsf{Vf})$. We describe each algorithm in turn, and prove security in Section 3.3.

$\mathsf{KeyGen}(1^\lambda, n, t) \to (\mathsf{pk}, \mathsf{cpk}, \mathsf{sk}_1, \ldots, \mathsf{sk}_n)$. Here pk is the signature verification public key, cpk is the combiner's public key, and $\mathsf{sk}_1, \ldots, \mathsf{sk}_n$ are the secret keys of the signing parties. The algorithm works as follows:

- First, run the threshold PRF $\mathsf{KeyGen}(1^\lambda, n, t)$ to get msk and $\mathsf{sk}_1, \ldots, \mathsf{sk}_n$. Also sample a key k in $\mathcal{K}$.
- Second, compute the elements $x_1, \ldots, x_\ell \in \mathcal{Y}$ on the bottom row of Figure 2 by setting $x_i \leftarrow F\big(\mathsf{msk}, (i, 0)\big)$ for $i \in [\ell]$.
- Next, generate the remaining elements of the rectangle in Figure 2 from $x_1, \ldots, x_\ell$. We denote entry (i, j) in the rectangle by $y[i, j]$ for $i \in [\ell]$ and $j \in [d-1]$. For example, $y[1, 1] = f(x_1)$ and $y[9, 3] = y_9$ in Figure 2.
- Set $\mathsf{pk} \leftarrow \Big(H\Big(k, y[1, d-1], \ldots, y[\ell, d-1]\Big),\ k\Big) \in \mathcal{Z} \times \mathcal{K}$, the hash of the top row.
- Finally, the combiner's public key $\mathsf{cpk} \in \mathcal{Y}^{\ell \times (d-1)}$ contains the rectangle of values from Figure 2 excluding the bottom row, but where each value, except the top row, is masked by $F(\mathsf{msk}, (i, j))$. That is, for $i = 1, \ldots, \ell$

$$\mathsf{cpk}[i, j] := \begin{cases} y[i, j] \oplus F\big(\mathsf{msk}, (i, j)\big) & \text{for } j = 1, 2, \ldots, d-2, \\ y[i, j] & \text{for } j = d-1. \end{cases}$$

- Output $(\mathsf{pk}, \mathsf{cpk}, \mathsf{sk}_1, \ldots, \mathsf{sk}_n)$.

$\mathsf{Sign}(\mathsf{sk}_k, m) \to \sigma_k$. Party $k \in [n]$ computes $P(m) = (j_1, \ldots, j_\ell) \in [0, d)^\ell$ and outputs the signature share σ_k which is an ℓ-tuple of PRF evaluation shares defined as

$$\sigma_k[i] \leftarrow \mathsf{Eval}\big(\mathsf{sk}_k,\ (i, j_i),\ m\big) \quad \text{for } i = 1, \ldots, \ell.$$

Note that the message m is used as the evaluation context in the threshold PRF evaluation. This ensures that the combiner will obtain $F\big(\mathsf{msk}, (i, j_i)\big)$ only if it has t evaluation shares obtained from signing a common message. It is not possible to combine evaluation shares for $F\big(\mathsf{msk}, (i, j_i)\big)$ from signing different messages (see Remark 3).

$\mathsf{Combine}\big(\mathsf{cpk},\ m,\ \mathcal{J} \in \binom{[n]}{t},\ \{\sigma_k\}_{k\in\mathcal{J}}\big) \to \sigma$. The combiner computes $P(m) = (j_1, \ldots, j_\ell) \in [0, d)^\ell$ and uses the provided PRF evaluation shares $\{\sigma_k\}_{k\in\mathcal{J}}$ to compute $F\big(\mathsf{msk}, (i, j_i)\big)$ for all $i \in [\ell]$. It outputs the signature $\sigma \in \mathcal{Y}^\ell$ by setting for $i = 1, \ldots, \ell$:

$$\sigma[i] \leftarrow \begin{cases} F\big(\mathsf{msk}, (i, j_i)\big) & \text{if } j_i = 0, \\ \mathsf{cpk}[i, j_i] \oplus F\big(\mathsf{msk}, (i, j_i)\big) & \text{if } 1 \le j_i \le d-2, \\ \mathsf{cpk}[i, j_i] & \text{if } j_i = d-1. \end{cases}$$

As promised, the final signature σ is a regular Winternitz signature.

$\mathsf{Vf}(\mathsf{pk}, m, \sigma) \to \{0, 1\}$. The verification algorithm is identical to WOTS signature verification.

Remark 2 (the size of cpk). The combiner's public key, cpk, contains $\ell \times (d-1)$ elements in $\mathcal{Y}$. As an example, Winternitz in SLH-DSA uses $d = 16$ and $\ell \le 67$.

Remark 3 (the need for PRF context). In the signing algorithm every party uses m as the evaluation context for the threshold PRF (the evaluation context for a threshold PRF is defined in Section 2). Without the evaluation context the scheme would be insecure. Consider the case of Lamport signatures, so that $d = 2$. Let m, m_1, m_2 be three messages such that $P(m) \ge \min(P(m_1), P(m_2))$, meaning that in every coordinate $P(m)$ is at least one of $P(m_1)$ or $P(m_2)$. Now suppose $t \ge 3$ and the adversary requests $\lfloor t/2 \rfloor$ signature shares for m, and $\lceil t/2 \rceil$ signature shares for each of m_1 and m_2. That is, it requests about $1.5t$ signature shares in total, each share from a different signing party. The adversary makes no corruption queries, so that $f = 0$. After making these queries the adversary should be unable to generate a signature. However, without the evaluation context the adversary would have t shares for each of the pre-images that together reveal the signature on m. This violates security. The evaluation context prevents these shares from being combined ensuring that the attack fails, as a consequence of Theorem 3.1. Finally, we note that the attack can also be made to work with fewer signature shares, only $(4/3)t$, if the adversary can find four messages m, m_1, m_2, m_3 such that every 0-entry in $P(m)$ is also a 0-entry in two or more of $P(m_1), P(m_2), P(m_3)$. Algebraically we can write this condition as

$$P(m) \ge \min\Big(\max\big(P(m_1), P(m_2)\big),\ \max\big(P(m_1), P(m_3)\big),\ \max\big(P(m_2), P(m_3)\big)\Big).$$

The adversary requests $\lfloor t/3 \rfloor$ signature shares for m, and $\lceil t/3 \rceil$ signature shares for each of m_1, m_2, m_3, from different signing parties, and obtains a signature for m. Again, this violates security.

Remark 4 (the scheme with P as a TCR). We described the threshold signature scheme assuming the function P is collision resistant. There is a strong desire to relax P to be only enhanced target collision resistant (eTCR) [26] so that its output length can be cut in half. To do so we would make P a keyed hash function $P : \mathcal{K} \times \mathcal{M} \to [0, d)^\ell$. When signing a message m the signer would choose a fresh random key k' in $\mathcal{K}$ and include k' in the signature. It would then sign m using the vector $P(k', m)$. In the setting of a threshold signature, choosing such a random key complicates the scheme since all t parties need to be involved in choosing this random k'. A simple way to do this adds one round to the signing protocol. We expand the secret key of every party i by also

The Game $\mathbf{G}^{\text{tsig}}_{\mathcal{A},\mathcal{S}}(\lambda)$

1 : $Q \leftarrow \emptyset$ // set of queried messages
2 : $D \leftarrow \emptyset$ // set of corrupt parties
3 : $(n, t, \mathsf{state}) \leftarrow\!\!\$\ \mathcal{A}(1^\lambda)$
4 : $(\mathsf{pk}, \mathsf{cpk}, \mathsf{sk}_1, \ldots, \mathsf{sk}_n) \leftarrow\!\!\$\ \mathsf{KeyGen}(1^\lambda, n, t)$
5 : $(m, \sigma) \leftarrow\!\!\$\ \mathcal{A}^{\mathsf{OCorrupt},\mathsf{OSign}}(1^\lambda, \mathsf{pk}, \mathsf{cpk}, \mathsf{state})$
6 : **if** $\mathsf{Vf}(\mathsf{pk}, m, \sigma) = 0$: **return** 0
// fail if too many sig. shares for m were exposed
7 : **if** $\left|D \cup Q[m]\right| \geq t$: **return** 0
// we require a sufficiently large t
8 : **if** $t \leq (n + \left|D\right|)/2$: **return** 0
9 : **return** 1

Oracle $\mathsf{OCorrupt}(i)$

1 : $D \leftarrow D \cup \{i\}$
2 : **return** sk_i

Oracle $\mathsf{OSign}(i, m)$

// honest parties sign at most one message
1 : **if** $\exists m'$ s.t. $i \in Q[m']$: **return** $\perp$
2 : $Q[m] \leftarrow Q[m] \cup \{i\}$
3 : **return** $\mathsf{Sign}(1^\lambda, \mathsf{sk}_i, m)$

Figure 3: Threshold one-time signature security game for a scheme $\mathcal{S}$ with algorithms $(\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Combine}, \mathsf{Vf})$.

giving it a key sk''_i for a threshold verifiable random function (threshold VRF) $\mathrm{VRF} : \mathcal{K} \times \mathcal{M} \to \mathcal{K}$. When signing a message m, each party would first send the VRF evaluation share $\mathrm{VRF}(sk''_i, m)$ to the combiner. The combiner would send the resulting key $k' \in \mathcal{K}$ along with a VRF proof of correct evaluation to the signing parties. Each party would verify the VRF proof, and if valid, would sign using the vector $P(k', m)$ as before. This two-round protocol lets us use an eTCR function P instead of a collision resistant P. Of course one can also use a commit-reveal protocol to generate k', making the overall signing protocol three rounds.

3.2 Security of threshold one-time signatures

We now turn to the security analysis of the threshold Winternitz scheme from Section 3.1. To prove security of the scheme we must first define security for a threshold one-time signature (OTS). The security game is presented in Figure 3. In this game, every honest party will only issue a signature for at most one message. As explained in the introduction, if the signing parties do not communicate with each other, then an OTS scheme like this can only be secure when the threshold t satisfies $t > (n + f)/2$, where f is the maximum number of corrupt parties. This is enforced on Line (8). This line ensures that the adversary can obtain at most one message-signature pair from the challenger (for example, by requesting t signature shares for that message).

Definition 3.2. A threshold one-time signature scheme $\mathcal{S} = (\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Combine}, \mathsf{Vf})$ is **adaptively secure** if for all PPT adversaries $\mathcal{A}$ the following advantage is negligible

$$\mathsf{Adv}^{\text{tsig}}_{\mathcal{A},\mathcal{S}}(\lambda) := \Pr\left[\mathbf{G}^{\text{tsig}}_{\mathcal{A},\mathcal{S}}(\lambda) = 1\right].$$

If $\mathcal{A}$ must issue all its queries to the oracle $\mathsf{OCorrupt}(\cdot)$ at the beginning of the game then we say that $\mathcal{S}$ is **statically secure**.

3.3 Security of our construction

To prove security we need iterates of the function $f : \mathcal{Y} \to \mathcal{Y}$ to be second preimage resistant. This is captured in the following definition. Recall from the notation discussion in the introduction that f should be understood to be an ensemble of functions indexed by the security parameter λ.

Definition 3.3. Let $d \geq 1$. A function $f : \mathcal{Y} \to \mathcal{Y}$ is **second preimage resistant (SPR) on d-iterates**, if for every PPT adversary $\mathcal{A}$ there is a negligible function ϵ_f such that

$$\mathsf{Adv}^{\mathrm{spr}}_{\mathcal{A},f,d}(\lambda) := \Pr\left[f^{(d-j+1)}(x') = f^{(d)}(x) \quad : \quad \begin{array}{l} x \leftarrow\!\$\, \mathcal{Y}, \quad j \leftarrow\!\$\, [d], \\ y \leftarrow f^{(j)}(x), \\ x' \leftarrow \mathcal{A}(1^\lambda, y, j) \end{array} \right] \leq \epsilon_f(\lambda)$$

where $f^{(1)} = f$ and $f^{(n)} = f(f^{(n-1)})$ for $n \geq 2$.

Here the adversary is given $y = f^{(j)}(x)$ and $j \in [d]$ as input and needs to find an x' such that either (i) $f(x') = y$, meaning that $\mathcal{A}$ found a preimage of y, or (ii) $f(x') \neq y$ and

$$f^{d-j}(f(x')) = f^{(d-j+1)}(x') = f^{(d)}(x) = f^{(d-j)}(y),$$

meaning that $f(x')$ is a second preimage for y with respect to the function $f^{(d-j)}$. When $d = 1$ this reduces to the standard one-way security game for f. However for $d \geq 2$ this requirement is stronger than standard one-wayness and standard second preimage resistance. We note that if $f : \mathcal{Y} \to \mathcal{Y}$ is modeled as a random oracle, then for small d, say $d < \lambda$, a q-query adversary $\mathcal{A}^f(y, j)$ will have advantage at most $qd/|\mathcal{Y}|$.

The following theorem states the security property of the construction $\mathcal{S}$ from Section 3.1.

Theorem 3.1. *The one-time threshold signature scheme $\mathcal{S}$ in Section 3.1 with parameters (ℓ, d) is adaptively secure assuming the underlying threshold PRF is adaptively secure, the function P is collision resistant and cover free (Definition 3.1), the function f is SPR on $(d-1)$-iterates (Definition 3.3), and the hash function H is target collision resistant.*

Proof. We prove the theorem in a sequence of games.

Game 0. This is the game $\mathbf{G}^{\mathrm{tsig}}_{\mathcal{A},\mathcal{S}}(\lambda)$ from Figure 3 applied to the one-time threshold signature scheme $\mathcal{S}$ in Section 3.1.

Game 1. The game $\mathbf{G}_1(\lambda)$ is the same as $\mathbf{G}^{\mathrm{tsig}}_{\mathcal{A},\mathcal{S}}(\lambda)$ except that we add two more failure conditions. Let (m, σ) be the final forgery from the adversary. The game returns 0 if

- there is some $m' \neq m$ such that $P(m') = P(m)$ and $|D \cup Q[m']| \geq t$, or
- the top row of the Winternitz table derived from σ is different from the top row generated during KeyGen.

The first condition means that the adversary forged a signature on m by finding a P collision with a message m' for which it was given a signature. This breaks the collision resistance of P. The second condition means that the adversary broke the target collision resistance of H. Let E_1 be the event that either of these conditions holds. Since we assume that P is collision resistant and H is target collision resistant, there are negligible functions ϵ_P and ϵ_H such that

$$\begin{aligned} \Big|\Pr\Big[\mathbf{G}^{\mathrm{tsig}}_{\mathcal{A},\mathcal{S}}(\lambda) = 1\Big] - \Pr\Big[\mathbf{G}_1(\lambda) = 1\Big]\Big| = \\ \Big|\Pr\Big[\mathbf{G}^{\mathrm{tsig}}_{\mathcal{A},\mathcal{S}}(\lambda) = 1\Big] - \Pr\Big[\mathbf{G}^{\mathrm{tsig}}_{\mathcal{A},\mathcal{S}}(\lambda) = 1 \ \wedge \ \neg\mathsf{E}_1\Big]\Big| \leq \Pr[\mathsf{E}_1] \leq \\ \epsilon_P(\lambda) + \epsilon_H(\lambda). \end{aligned} \tag{1}$$

Game 2. Game $\mathbf{G}_2(\lambda)$ is the same as $\mathbf{G}_1(\lambda)$ except that at the beginning of the game we sample a pair $(i^*, j^*) \leftarrow\!\!\$\ [\ell] \times [d-1]$. At the end of the game we add one more failure condition E_2 defined as follows. Let (m, σ) be the final forgery from the adversary. Let (m', σ') be the single message-signature pair for which the adversary was able to obtain t shares from distinct parties using the same context (if there is no such message then choose m' arbitrarily in $\mathcal{M} \setminus \{m\}$). Then E_2 happens exactly when

$$P(m)[i^*] < j^* \leq P(m')[i^*].$$

In other words, on chain i^* in the Winternitz table, the value j^* falls between the forgery σ and the obtained signature σ'. When E_2 does not happen, the game returns 0.

Clearly (i^*, j^*) are independent of the adversary's view. Moreover, because P is cover-free, there must exist at least one chain i where $P(m)[i] < P(m')[i]$. It follows that $\Pr[\mathsf{E}_2] \geq 1/(\ell d)$. This shows that

$$\begin{aligned} \Pr\Big[\mathbf{G}_2(\lambda) = 1\Big] = \Pr\Big[\mathbf{G}_1(\lambda) = 1 \ \wedge\ \mathsf{E}_2\Big] = \\ \Pr\Big[\mathbf{G}_1(\lambda) = 1\Big] \cdot \Pr\big[\mathsf{E}_2\big] \geq \Pr\Big[\mathbf{G}_1(\lambda) = 1\Big]/(\ell d). \end{aligned} \tag{2}$$

Game 3. Game $\mathbf{G}_3(\lambda)$ is the same as $\mathbf{G}_2(\lambda)$, but we replace a few evaluations of the threshold PRF with truly random values. In particular, when generating the combiner's public key cpk, the bottom j^* elements on chain i^* are sampled at random instead of them being generated as in regular key generation. More precisely, during key generation the elements in column i^* of cpk are generated as follows: sample j^* values $x, y_1, \ldots, y_{j^*-1} \leftarrow\!\!\$\ \mathcal{Y}$ and for $j \in [d]$ set

$$\mathsf{cpk}[i^*, j] := \begin{cases} f^{(j)}(x) \oplus y_j & \text{if } 1 \leq j < j^*, \\ f^{(j)}(x) \oplus F\big(\mathsf{msk}, (i^*, j)\big) & \text{if } j^* \leq j < d-1, \\ f^{(j)}(x) & \text{if } j = d-1. \end{cases}$$

The remaining columns of cpk are generated as in Game 2.

Say the adversary corrupts u parties. When E_2 happens, we know that for $j < j^*$, the adversary obtains from the challenger fewer than $t - u$ PRF evaluation shares under a common evaluation context for a point (i^*, j). Consequently $F\big(\mathsf{msk}, (i^*, j)\big)$ is indistinguishable from random for all $j < j^*$. More precisely, if the adversary could distinguish Games 2 and 3 then it could be used to attack the threshold PRF with the same advantage. Since we assume that F is a secure context-aware threshold PRF, there is a negligible function ϵ_F such that

$$\Big|\Pr\Big[\mathbf{G}_2(\lambda) = 1\Big] - \Pr\Big[\mathbf{G}_3(\lambda) = 1\Big]\Big| \leq \epsilon_F(\lambda). \tag{3}$$

Breaking the second preimage resistance of f. Finally, we show that if the adversary $\mathcal{A}$ has non-negligible probability of winning in Game $\mathbf{G}_3(\lambda)$ then there is an adversary $\mathcal{B}$, with about the same running time as $\mathcal{A}$, that breaks the $(d-1)$-iterated SPR of f with the same probability. Hence, since we assume that f is SPR on $(d-1)$ iterates, there must be a negligible function ϵ_f such that

$$\Pr\Big[\mathbf{G}_3(\lambda) = 1\Big] \leq \epsilon_f(\lambda). \tag{4}$$

Algorithm $\mathcal{B}(y^*, j^*)$ for breaking $(d-1)$-iterated SPR for f as in Definition 3.3 operates as follows. The algorithm takes as input a random $j^* \in [d-1]$ and $y^* = f^{(j^*)}(x^*)$ for $x^* \leftarrow\!\!\$\ \mathcal{Y}$. It needs

to respond with some x' such that $f^{(d-j^*)}(x') = f^{(d-1)}(x^*)$. To do so, $\mathcal{B}$ runs the key generation algorithm of Game 3 with a few differences. First, as in Game 3, it samples a random $i^* \leftarrow\!\!\$\ [\ell]$, but instead of sampling j^* in $[d-1]$ itself, it uses the j^* provided to it as input. Next, it constructs cpk as follows. In all columns other than i^* it constructs cpk as in Game 3 key generation. In column i^* it constructs cpk as follows: sample $y_1, \ldots, y_{j^*-1} \leftarrow\!\!\$\ \mathcal{Y}$ and set

$$\mathsf{cpk}[i^*, j] := \begin{cases} y_j & \text{if } 1 \le j < j^*, \\ f^{(j-j^*)}(y^*) \oplus F\big(\mathsf{msk}, (i^*, j)\big) & \text{if } j^* \le j < d-1, \\ f^{(j-j^*)}(y^*) & \text{if } j = d-1. \end{cases}$$

Since $y^* = f^{(j^*)}(x^*)$, this cpk is distributed exactly as in Game 3. Our $\mathcal{B}$ then runs $\mathcal{A}$ as in Game 3. Since we assume that event E_2 happens, our $\mathcal{B}$ can respond to all of $\mathcal{A}$'s corruption and signing queries. Moreover, when $\mathcal{A}$ outputs a forgery (m, σ) we know that $j' := P(m)[i^*]$ is less than j^*. Let $y' := \sigma[i^*]$. Since (m, σ) is a valid message-signature pair then

$$f^{(d-j^*)}\big(f^{(j^*-j'-1)}(y')\big) = f^{(d-1-j')}(y') = f^{(d-1)}(x^*). \tag{5}$$

The equality on the right holds because $f^{(d-1)}(x^*)$, which $\mathcal{B}$ can compute as $f^{(d-1-j^*)}(y^*)$, is the element at the top row of column i^*. Since we assume that event E_1 does not happen, a valid forgery must satisfy the equality on the right. But then $x' := f^{(j^*-j'-1)}(y')$ is a valid solution to the given $(d-1)$-iterated SPR challenge since by (5) we have that $f^{((d-1)-j^*+1)}(x') = f^{(d-j^*)}(x') = f^{(d-1)}(x^*)$. Hence, (4) follows.

Completing the proof. Putting equations (1) – (4) together shows that

$$\Pr\big[\mathbf{G}^{\mathrm{tsig}}_{\mathcal{A},\mathcal{S}}(\lambda) = 1\big] \le \epsilon_P(\lambda) + \epsilon_H(\lambda) + \ell d \cdot \big(\epsilon_F(\lambda) + \epsilon_f(\lambda)\big) \tag{6}$$

which is a negligible function, as required. This completes the proof of Theorem 3.1. □

Remark 5. The notion of SPR on d-iterates (Definition 3.3) can also be used to analyze the non-threshold WOTS signature scheme. However, as in (6), the reduction to the SPR assumption will incur a factor of (ℓd) loss.

3.4 Applications beyond WOTS

In this section we briefly explain how to apply the method we used to thresholdize WOTS to other hash-based signature schemes. This section assumes knowledge of how these schemes work. Recall that the combiner holds no secrets and its only job is to combine the provided signature shares into a complete signature. In particular, its key cpk is assumed to be known to the adversary, as shown in Figure 3.

XMSS. The XMSS (stateful) signature scheme places several WOTS public keys in the leaves of a Merkle tree. To thresholdize the scheme, we would apply our construction to each of the WOTS instances. In this case, the size of the combiner's public key, cpk, is linear in the number of WOTS instances used. Since cpk includes the WOTS public key, the combiner can construct the required Merkle proofs on its own.

Multi-tree XMSS. XMSS-MT is a stateful signature scheme where the signer uses a tree of XMSS trees. Each internal XMSS leaf is used to sign the root of an XMSS tree just below it. The leaves of the overall structure are used to sign messages. This lets one sign a large number of messages with a single public key, using the leaves one at a time from left to right, while keeping key generation time low. This XMSS-MT scheme can be thresholdized using generic multiparty computation (MPC). However, the MPC would need to run for every message being signed. Our approach lets us amortize the cost of the MPC. As the signers move from left to right in the overall structure they do the following. First, every time the signers jump from one XMSS tree to the next, the signers will run an MPC to generate the combiner's public key cpk for the next XMSS tree, and erase the cpk for the current used-up tree. The next set of signatures will all be generated using our approach (i.e. with no MPC) and the newly generated cpk. When this XMSS tree is exhausted, another MPC will run to generate cpk for the next XMSS tree, and so on. In this way, the MPC only needs to run once every few hundred signatures, and message signing time is fast. We view this as a hybrid MPC-PRAWNS approach.

HORS, FORS, and PORS. A context-aware threshold PRF can be used to thresholdize the HORS [43], FORS [32], and PORS [3, 1] signature schemes in a similar way to how we thresholdize WOTS. However, unlike WOTS, the HORS and FORS schemes do not use a hash chain, or more precisely, they use a hash chain of length one (namely $d = 1$). As a result, the vector of secret preimages is directly generated by the threshold PRF, and the combiner's public key cpk contains no masked hashed values. In the FORS scheme, a Merkle tree is used to authenticate the preimages provided in the signature. The combiner will generate these Merkle proofs on its own, and to do so, cpk will include all the (public) leaves of this FORS Merkle tree. Hence, the size of cpk is linear in the total number of leaves in the FORS forest.

SLH-DSA. SLH-DSA [40] is a combination of FORS, WOTS and XMSS. The NIST standard is designed to support 2^{64} signatures per public key. The scheme uses a tree with about 2^{64} leaves, where each leaf is used to sign one FORS public key. Recently, the Sphincs$^-$ effort defined a number of reduced parameter sets for Sphincs$^+$ designed to shrink the signature size and verification time, at the cost of supporting fewer than 2^{64} stateless signatures per public key. NIST adopted one of these reduced parameter sets into what is now called the SLH-DSA-24 family [18] that supports 2^{24} stateless signatures per public key.

Our approach is not designed for the Sphincs$^+$ framework as that would require materializing all the FORS secret keys and their derived Merkle forests during key generation. This is too costly and would result in a prohibitively large cpk. Moreover, if the key generator were forced to materialize all the FORS secret keys, then we might as well treat this set of secret keys as a single large FORS scheme, and avoid the Sphincs$^+$ framework altogether.

Shrincs. The Shrincs signature scheme [15], designed for a post-quantum Bitcoin, uses both a stateless signature (a scaled down SLH-DSA) and a stateful signature (XMSS). Under normal operations, the user's wallet always signs with the short stateful scheme. However, if the state is lost or corrupt, the user can sign using the stateless scheme, and re-initialize a new stateful public key. The stateless scheme is parameterized to support up to 2^{40} stateless signings.

Our techniques directly apply to thresholdize the stateful part of Shrincs, as discussed above. For the stateless part, the threshold scheme must generate a valid Sphincs$^+$ signature with the

Shrincs parameters. Thresholdizing this as is would result in a prohibitively large combiner's public cpk. However, a technique called hypertree pruning [14] lets the signer shrink the signature parameters while generating a valid Shrincs stateless signature. The idea is to have the signer try many randomizers until it finds one that uses a leaf in the Sphincs$^+$ hypertree that is contained in a small pre-determined subset of the tree. This pruning step ensures that only a small number of hypertree leaves are ever used for signing. Now the key generator only needs to materialize the FORS forests for those leaves, and this results in a manageable size cpk. The same pruning technique applies to multi-tree XMSS (XMSS-MT) and other systems based on Sphincs$^+$. It can be used to generate correctly formatted valid signatures, while supporting a manageable size cpk in our threshold scheme.

It remains to explain how to choose the randomizer used in hypertree prunning. We cannot trust one of the parties or the combiner to choose the randomizer, since they might bias all signatures towards a single leaf in the hypertree. This would be insecure. Instead, the quorum of t signing parties would use a commit-reveal protocol to sample a random PRF key known to all t parties. Then they each locally search for the smallest PRF input so that the PRF output is a randomizer that satisfies the requirements. As long as at least one of the t signers is honest, the result is a randomizer that is indistinguishable from a uniform randomizer that satisfies the requirements.

4 A lattice-based context-aware threshold PRF

We now turn to constructing a context-aware threshold PRF. We give two constructions. In this section we give a lattice-based construction. In the next section we give a hash-based construction. We stress that the PRF lattice construction is only used during threshold signing. The signature verifier continues to verify a regular Winternitz signature.

We make the following two assumptions to simplify the schemes. First, both constructions, as described, are not designed to detect a party that is trying to actively disrupt the PRF evaluation process. This form of robustness can be achieved using leveled homomorphic signatures [25], as was suggested in previous works on threshold lattice-based cryptography [8]. We refer to [33, App. E] for the details. Alternatively, robustness can be achieved by having every party attach a NIZK proof to prove that its contribution is constructed correctly.

Second, in both constructions we slightly modify the threshold PRF syntax from Section 2: we allow the PRF Eval algorithm to also take the quorum $\mathcal{J}$ as input. The signing algorithm in the derived threshold signature scheme would then also take the quorum $\mathcal{J}$ as input. The security game for a threshold PRF in Figure 1 is similarly modified so that the oracle OEval takes $\mathcal{J}$ as input and passes it to the Eval algorithm. Nothing else changes. We stress that allowing Eval to take the quorum as input is an efficiency measure that lets us build more efficient PRFs. It is not a security measure. In particular, the quorum $\mathcal{J}$ cannot be used in place of the evaluation context ec. Querying for t shares of the form $\mathsf{Eval}(\mathsf{sk}_i, \mathcal{J}_i, x, \mathsf{ec})$ may reveal information about the PRF value at x, despite being generated with respect to different quorums $\mathcal{J}_1, \ldots, \mathcal{J}_t$.

A lattice PRF. To construct our lattice-based threshold PRF let us first review a simple lattice-based PRF described in [9]. The PRF is parameterized by two primes $p < q$, an integer d, and a hash function $H : \mathcal{X} \to \mathbb{Z}_q^d$. The PRF uses a random secret key $\mathsf{msk} \in \mathbb{Z}_q^d$ and is defined as

$$F_{\text{LWR}}(\mathsf{msk}, x) := \lfloor \langle H(x), \mathsf{msk} \rangle \rceil_p \in \mathbb{Z}_p \tag{7}$$

where $\langle \cdot, \cdot \rangle$ denotes inner product, and $\lfloor b \rceil_p$ denotes rounding an element $b \in \mathbb{Z}_q$ to $\mathbb{Z}_p$ by lifting b to the integers in $[0, q-1]$, multiplying by (p/q), rounding the result to the nearest integer, and reducing modulo p. This can be shown to be a secure PRF when H is modeled as a random oracle, assuming the lattice with rounding (LWR) [5] assumption holds.

Libert, Stehlé, and Titiu [33] constructed an elegant adaptively-secure threshold PRF based on F_{LWR} using a lossy trapdoor function. Their scheme is context free (i.e., not context aware). They use a linear integer secret sharing scheme (LISS) due to Damgård and Thorbek [17] to support monotone access structures that can be implemented by a low-depth monotone formula. This LISS supports small reconstruction constants, where the final PRF value is recovered from a low-norm linear combination of evaluation shares, which keeps the norm of the noise small. They then propose to implement a threshold access structure using Valiant's monotone formula for the majority function [46], and its improvement by Hoory, Magen, and Pitassi [27], that gives a formula of size $\tilde{O}(n^{1+\sqrt{2}})$ and depth $O(\log n)$, for n inputs. The resulting average share size for each of the n parties is then $\tilde{O}_\lambda(n^{\sqrt{2}})$. More recently, [44] proposed another (context free) threshold PRF based on F_{LWR} that works when $\binom{n}{t}$ is polynomial size.

A context-aware threshold lattice PRF. We construct a context-aware threshold PRF, called F_{lat}, from the non-threshold PRF in (7). Instead of using LISS as in [33], we use standard Shamir secret sharing. We handle the noise growth in the proof of security caused by the Lagrange coefficients by "clearing out the denominators" and using sufficiently large LWE parameters (as done in [8] for threshold FHE). This results in a scheme with key share size $O_\lambda(n \log n)$ per party for n parties. The key share size can be reduced to $O_\lambda(n)$ per party using a clever choice of Lagrange interpolation points due to Brakerski et al. [11]. It is a practical choice for, say, a 30-out-of-40 scheme, where $\binom{n}{t}$ is too large for the combinatorial schemes. However, we view context-awareness as the main contribution.

Our construction is parameterized by two primes $p < q$, an integer d, an upper bound N on the number of parties, and two hash functions $H : \mathcal{X} \to \mathbb{Z}_q^d$ and $H' : \mathcal{X} \times \mathcal{E} \times \binom{[n]}{t} \to \mathbb{Z}_q^d$, where $\mathcal{E}$ is the context space. The context-aware threshold PRF works as follows.

- $\mathsf{KeyGen}(1^\lambda, n, t) \to (\mathsf{msk}, \mathsf{sk}_1, \ldots, \mathsf{sk}_n)$. For $t \le n \le N$ the algorithm does
  - sample msk in $\mathbb{Z}_q^d$ and construct a t-of-n Shamir secret sharing of msk to obtain shares $k_1, \ldots, k_n \in \mathbb{Z}_q^d$,
  - sample a t-of-n Shamir secret sharing of the value $0^d \in \mathbb{Z}_q^d$ to obtain shares $k'_1, \ldots, k'_n$ in $\mathbb{Z}_q^d$,
  - set $\mathsf{sk}_i \coloneqq (i, k_i, k'_i)$ for $i = 1, \ldots, n$,
  - output $(\mathsf{msk}, \mathsf{sk}_1, \ldots, \mathsf{sk}_n)$.
- $\mathsf{Eval}(\mathsf{sk}_i,\ \mathcal{J} \in \binom{[n]}{t},\ x \in \mathcal{X},\ \mathsf{ec}) \to y_i$. Takes as input a quorum $\mathcal{J}$ with $i \in \mathcal{J}$, an evaluation point $x \in \mathcal{X}$, and an evaluation context $\mathsf{ec} \in \mathcal{E}$. It does
  - compute the i-th Lagrange coefficient $\lambda_i \in \mathbb{Z}_q$ for the quorum $\mathcal{J} \in \binom{[n]}{t}$, that is, $\lambda_i \coloneqq \prod_{j \in \mathcal{J} \setminus \{i\}} (0 - j)/(i - j)$.
  - compute $mask_i \leftarrow \lambda_i \cdot \langle H'(x, \mathsf{ec}, \mathcal{J}), k'_i \rangle \in \mathbb{Z}_q$,
  - compute $z_i \leftarrow \lambda_i \cdot \langle H(x), k_i \rangle \in \mathbb{Z}_q$,

– output $y_i \leftarrow \lfloor z_i \rceil_p + \lfloor mask_i \rceil_p \in \mathbb{Z}_p$.

- $\mathsf{Combine}(\mathcal{J} \in \binom{[n]}{t},\ x,\ \{y_i\}_{i\in\mathcal{J}}) \to y$. Output $y \leftarrow \sum_{i\in\mathcal{J}} y_i$ in $\mathbb{Z}_p$.

Correctness. We show that $\mathsf{Combine}$ outputs an approximately correct value, namely a value that is close to $F_{\mathrm{LWR}}(\mathsf{msk}, x)$. Since $\sum_{i\in\mathcal{J}} \lambda_i \cdot k_i = \mathsf{msk}$ and $\sum_{i\in\mathcal{J}} \lambda_i \cdot k'_i = 0$ it should be clear that

$$\left\lfloor \sum_{i\in\mathcal{J}} z_i \right\rceil_p = F_{\mathrm{LWR}}(\mathsf{msk}, x) \quad \text{and} \quad \sum_{i\in\mathcal{J}} mask_i = 0.$$

Therefore there is Δ in $\mathbb{Z}_p$, where $-t \le \Delta \le t$, such that

$$\sum_{i\in\mathcal{J}} y_i = \sum_{i\in\mathcal{J}} \lfloor z_i \rceil_p + \sum_{i\in\mathcal{J}} \lfloor mask_i \rceil_p = \left\lfloor \sum_{i\in\mathcal{J}} z_i \right\rceil_p + \left\lfloor \sum_{i\in\mathcal{J}} mask_i \right\rceil_p + \Delta = F_{\mathrm{LWR}}(\mathsf{msk}, x) + \Delta$$

The bound $-t \le \Delta \le t$ is due to the fact that for every $a_1, \ldots, a_t \in \mathbb{Z}_q$ the sum $(\lfloor a_1 \rceil_p + \ldots + \lfloor a_t \rceil_p)$ is at most $t/2$ away from $\lfloor a_1 + \ldots + a_t \rceil_p$ in $\mathbb{Z}_p$.

The end result is that the output of the combine algorithm can be up to t away from the correct PRF value. Hence, the combiner knows that one of $\{y-t, \ldots, y+t\}$ in $\mathbb{Z}_p$ is the correct PRF value. When assembling the final signature, the combiner can test all $2t+1$ candidates until it obtains the correct value. Recall that the combiner's public key cpk includes the top row of the rectangle in Figure 2, so that the combiner can tell when it found the correct PRF value. To reduce the number of candidates to test, we can modify the PRF and define its output as the result of chopping off the bottom $\lceil \log_2(t) \rceil$ bits of the output of F_{LWR} from (7). The combiner could then similarly chop off the bottom $\lceil \log_2(t) \rceil$ bits of each of the $2t+1$ candidates so that the list of candidates becomes only three or four candidates. Now when assembling the final signature, the combiner would only need to test these three or four candidates.

4.1 Security from LWE

We prove security of the scheme using the learning with errors (LWE) assumption [42]. Since in this paper n refers to the number of signing parties, we will use the variable d for what is usually denoted by n in lattice cryptography.

Definition 4.1 (LWE)**.** Let $q = q(\lambda)$ be a prime, let $d = d(\lambda)$ and $m = m(\lambda)$ be integers with $m > d$, and let $\chi = \chi(\lambda)$ be a distribution over $\mathbb{Z}$. The **LWE assumption** holds for parameters q, χ, d, m if for every PPT algorithm $\mathcal{A}$ the following function is negligible

$$\mathsf{Adv}^{\mathrm{lwe}}_{\mathcal{A},q,\chi,d,m}(\lambda) := \Big|\Pr\Big[\mathcal{A}(\mathbf{A}, \mathbf{As}+\mathbf{e}) = 1\Big] - \Pr\Big[\mathcal{A}(\mathbf{A}, \mathbf{r}) = 1\Big]\Big|,$$

where $\mathbf{A} \leftarrow\!\$\ \mathbb{Z}_q^{m\times d},\ \mathbf{s} \leftarrow\!\$\ \mathbb{Z}_q^d,\ \mathbf{e} \leftarrow\!\$\ \chi^m,\ \mathbf{r} \leftarrow\!\$\ \mathbb{Z}_q^m$.

Definition 4.2. For a distribution $\chi = \chi(\lambda)$ over $\mathbb{Z}$, we let $B_\chi = B_\chi(\lambda)$ be a function whose range is $\mathbb{Z}$ so that

$$\Pr\big[|\mathbf{e}| > B_\chi \ :\ \mathbf{e} \leftarrow\!\$\ \chi\big] < \mathsf{negl}(\lambda). \tag{8}$$

We will also need parameters so that the following statistical distance is negligibly small.

The Game $\mathbf{G}^{\text{pLWR}}_{\mathcal{A},(p,q,d,N)}(\lambda,\ b)$ for $b \in \{0,1\}$

1 : $(n, t, D, \mathsf{state}) \leftarrow\!\$\ \mathcal{A}(1^\lambda)$ // $\mathcal{A}$ returns $D \subseteq [n]$ where $|D| < t \le n \le N$
2 : $\mathbf{K} \leftarrow\!\$\ (\mathbb{Z}_q^{(<t)}[X])^d$ // sample a vector of polynomials of degree less than t
3 : **for** $\ell \in \mathcal{L}$: // $\mathcal{L}$ is some finite set of labels
4 : $\quad \mathbf{a}_\ell \leftarrow\!\$\ \mathbb{Z}_q^d$
5 : $\quad R_\ell \leftarrow\!\$\ \{\mathbb{Z}_q^{(<t)}[X]$ s.t. $\forall j \in D : R_\ell(j) = \langle \mathbf{a}_\ell, \mathbf{K}(j)\rangle\ \}$
6 : $b' \leftarrow\!\$\ \mathcal{A}^{\mathsf{Opoly},\mathsf{OH}}\big(1^\lambda, \{\mathbf{K}(j)\}_{j\in D}, \mathsf{state}\big)$
7 : **return** $b' \in \{0,1\}$

Oracle $\mathsf{Opoly}(i \in \mathcal{J} \cup \{0\},\ \mathcal{J} \in \binom{[n]}{t},\ \ell \in \mathcal{L})$

1 : **if** $i > 0 : \lambda_i^{\mathcal{J}} \leftarrow$ (Lagrange coefficient for i and $\mathcal{J}$)
2 : **if** $i = 0 : \lambda_i^{\mathcal{J}} \leftarrow 1$
3 : $y_{i,\ell} \leftarrow \big\lfloor \lambda_i^{\mathcal{J}} \cdot \langle \mathbf{a}_\ell, \mathbf{K}(i)\rangle \big\rceil_p$
4 : **if** $b = 1$: $y_{i,\ell} \leftarrow \big\lfloor \lambda_i^{\mathcal{J}} \cdot R_\ell(i) \big\rceil_p$
5 : **return** $y_{i,\ell} \in \mathbb{Z}_p$

Oracle $\mathsf{OH}(\ell \in \mathcal{L})$

1 : **return** $\mathbf{a}_\ell \in \mathbb{Z}_q^d$

Figure 4: The Poly-LWR games $\mathbf{G}^{\text{pLWR}}_{\mathcal{A},(p,q,d,N)}(\lambda, 0)$ and $\mathbf{G}^{\text{pLWR}}_{\mathcal{A},(p,q,d,N)}(\lambda, 1)$.

Definition 4.3. For primes $p = p(\lambda)$ and $q = q(\lambda)$ where $p < q$, let $\Delta_{p,q}(\lambda)$ denote the statistical distance between the uniform distribution over $\mathbb{Z}_p$ and the distribution

$$\mathcal{D}_{p,q}(\lambda) \coloneqq \Big\{ \lfloor x \rceil_p \ :\ x \leftarrow\!\$\ \mathbb{Z}_q \Big\}.$$

The statistical distance $\Delta_{p,q}(\lambda)$ is negligible when p/q is negligible. With these concepts in place, the following theorem reduces the security of our context-aware threshold PRF to the LWE assumption.

Theorem 4.1. *The threshold PRF $F_{lat} = (\mathsf{KeyGen}, \mathsf{Eval}, \mathsf{Combine})$ with parameters p, q, d, N is statically secure (as in Definition 2.2) assuming (i) the LWE assumption holds with parameters q, χ, d, m where $m = \mathsf{poly}(\lambda)$, (ii) $B_\chi N^{4N}(p/q)$ is negligible, (iii) the statistical distance $\Delta_{p,q}(\lambda)$ is negligible, and (iv) H and H' are modeled as random oracles.*

We prove the theorem in two steps. First, we define a new security property called Poly-LWR that generalizes the learning with rounding (LWR) assumption. We prove in Lemma 4.2 that Poly-LWR follows from the LWE assumption. Then in Lemma 4.3 we prove that F_{lat} is a secure context-aware threshold PRF assuming Poly-LWR holds. These two lemmas together prove Theorem 4.1.

Figure 4 defines the two security games for the Poly-LWR problem. In the $b = 0$ game queries to the oracle $\mathsf{Opoly}(\cdot, \cdot, \ell)$ are always answered with the same polynomial vector $\mathbf{K}$. In the $b = 1$ case queries to $\mathsf{Opoly}(\cdot, \cdot, \ell)$ are answered with a fresh independent polynomial R_ℓ for each $\ell \in \mathcal{L}$.

Definition 4.4 (Poly-LWR)**.** We say that the Poly-LWR assumption holds for parameters $p = p(\lambda), q = q(\lambda), d = d(\lambda), N = N(\lambda)$ if for every PPT adversary $\mathcal{A}$ the following advantage is a negligible function

$$\mathsf{Adv}^{\mathrm{plwr}}_{\mathcal{A},(p,q,d,N)}(\lambda) \coloneqq \left|\Pr\left[\mathbf{G}^{\mathrm{pLWR}}_{\mathcal{A},(p,q,d,N)}(\lambda,0)=1\right] - \Pr\left[\mathbf{G}^{\mathrm{pLWR}}_{\mathcal{A},(p,q,d,N)}(\lambda,1)=1\right]\right|,$$

where $\mathbf{G}^{\mathrm{pLWR}}_{\mathcal{A},(p,q,d,N)}(\lambda,0)$ and $\mathbf{G}^{\mathrm{pLWR}}_{\mathcal{A},(p,q,d,N)}(\lambda,1)$ are defined in Figure 4. Moreover, the following advantage is also a negligible function

$$\mathsf{Adv}^{\mathrm{plwr0}}_{\mathcal{A},(p,q,d,N)}(\lambda) \coloneqq \left|\Pr\left[\mathbf{G}^{\mathrm{pLWR0}}_{\mathcal{A},(p,q,d,N)}(\lambda,0)=1\right] - \Pr\left[\mathbf{G}^{\mathrm{pLWR0}}_{\mathcal{A},(p,q,d,N)}(\lambda,1)=1\right]\right|,$$

where $\mathbf{G}^{\mathrm{pLWR0}}_{\mathcal{A},(p,q,d,N)}(\lambda,b)$ is the same as $\mathbf{G}^{\mathrm{pLWR}}_{\mathcal{A},(p,q,d,N)}(\lambda,b)$, except that on lines (2) and (5) the sampled polynomials $\mathbf{K} \in (\mathbb{Z}_q^{(<t)}[X])^d$ and $R_\ell \in \mathbb{Z}_q^{(<t)}[X]$ are further required to evaluate to zero at $X = 0$.

Lemma 4.2. *The Poly-LWR assumption holds with parameters p, q, d, N assuming that LWE is hard with parameters q, χ, d, m where $m = \mathsf{poly}(\lambda)$, and $B_\chi N^{4N}(p/q)$ is a negligible function.*

We provide the proof of Lemma 4.2 in Appendix A.

Lemma 4.3. *The threshold PRF $F_{lat} = (\mathsf{KeyGen}, \mathsf{Eval}, \mathsf{Combine})$ with parameters p, q, d, N is statically secure (as in Definition 2.2) assuming (i) Poly-LWR holds for parameters p, q, d, N, (ii) $\Delta_{p,q}(\lambda)$ is negligible, and (iii) H and H' are modeled as random oracles.*

Proof. We prove the lemma using a sequence of five hybrid games. The core of the proof is the transition from Game 2 to Game 3.

Game 0. Game $\mathbf{G}_0(\lambda)$ is identical to $\mathbf{G}^{\mathrm{tprf}}_{\mathcal{A},F_{\mathrm{lat}}}(\lambda,0)$ from Figure 1. In this game the challenger first samples $\mathbf{K} \leftarrow\!\$\ (\mathbb{Z}_q^{(<t)}[X])^d$ and sets $\mathsf{msk} \leftarrow \mathbf{K}(0) \in \mathbb{Z}_q^d$. It then samples $\mathbf{K}' \leftarrow\!\$\ (\mathbb{Z}_q^{(<t)}[X])^d$ where $\mathbf{K}'(0) = 0$.

- It responds to a challenge query $\mathsf{OChallenge}(x)$ with $\lfloor \langle H(x), \mathsf{msk} \rangle \rceil_p \in \mathbb{Z}_p$.
- It responds to an evaluation query $\mathsf{OEval}(i, \mathcal{J}, x, \mathsf{ec})$ with $y_i + mask_i \in \mathbb{Z}_p$ where

$$y_i \leftarrow \lfloor \lambda_i \cdot \langle H(x), \mathbf{K}(i) \rangle \rceil_p \quad \text{and} \quad mask_i \leftarrow \lfloor \lambda_i \cdot \langle H'(x, \mathsf{ec}, \mathcal{J}), \mathbf{K}'(i) \rangle \rceil_p$$

  Here $\lambda_i \in \mathbb{Z}_q$ is the i-th Lagrange coefficient for the quorum $\mathcal{J}$.

Game 1. Game $\mathbf{G}_1(\lambda)$ is identical to Game $\mathbf{G}_0(\lambda)$ except that we tweak how the challenger responds to evaluation queries from $\mathcal{A}$. In particular, we change the masks to be the evaluation of random polynomials, consistent with the adversary's view. Let $D \subseteq [n]$, with $|D| < t$, be the set of corrupted parties fixed at the beginning of the game. For every triple $(\mathcal{J}, x, \mathsf{ec})$ the challenger samples a fresh polynomial

$$R'_{\mathcal{J},x,\mathsf{ec}} \leftarrow\!\$ \left\{ \mathbb{Z}_q^{(<t)}[X] \text{ s.t. } \forall j \in D : R'_{\mathcal{J},x,\mathsf{ec}}(j) = \langle H'(x, \mathsf{ec}, \mathcal{J}), \mathbf{K}'(j) \rangle \text{ and } R'_{\mathcal{J},x,\mathsf{ec}}(0) = 0 \right\}$$

It then responds to an evaluation query $\mathsf{OEval}(i, \mathcal{J}, x, \mathsf{ec})$ with $y_i + \widehat{mask}_i \in \mathbb{Z}_p$ where

$$y_i \leftarrow \lfloor \lambda_i \cdot \langle H(x), \mathbf{K}(i) \rangle \rceil_p \quad \text{and} \quad \widehat{mask}_i \leftarrow \lfloor \lambda_i \cdot R'_{\mathcal{J},x,\mathsf{ec}}(i) \rceil_p$$

By the Poly-LWR assumption with label set $\mathcal{L} \coloneqq \binom{[n]}{t} \times \mathcal{X} \times \mathcal{E}$, the adversary cannot distinguish Game 0 from Game 1. In particular,

$$\left|\Pr\Big[\mathbf{G}_0(\lambda) = 1\Big] - \Pr\Big[\mathbf{G}_1(\lambda) = 1\Big]\right| \leq \epsilon_{\text{pLWR0}}(\lambda) \tag{9}$$

for some negligible function ϵ_{pLWR0}.

Game 2. Game $\mathbf{G}_2(\lambda)$ is identical to Game $\mathbf{G}_1(\lambda)$ except that we further change how the challenger responds to evaluation and challenge queries from $\mathcal{A}$. In particular, for every $x \in \mathcal{X}$ we change the PRF value at x and its shares to be the evaluation of a random polynomial R_x, consistent with the adversary's view. To do so, at the beginning of the game the challenger further samples for every $x \in \mathcal{X}$ a fresh polynomial

$$R_x \leftarrow\!\!\$ \left\{ \mathbb{Z}_q^{(<t)}[X] \text{ s.t. } \forall j \in D : R_x(j) = \big\langle H(x), \mathbf{K}(j) \big\rangle \right\}$$

- The challenger responds to an evaluation query $\mathsf{OEval}(i, \mathcal{J}, x, \mathsf{ec})$ with $\hat{y}_i + \widehat{mask}_i \in \mathbb{Z}_p$ where

$$\hat{y}_i \leftarrow \big\lfloor \lambda_i \cdot R_x(i) \big\rceil_p \quad \text{and} \quad \widehat{mask}_i \leftarrow \big\lfloor \lambda_i \cdot R'_{\mathcal{J},x,\mathsf{ec}}(i) \big\rceil_p$$

- The challenger responds to a challenge query $\mathsf{OChallenge}(x)$ with $\lfloor R_x(0) \rceil_p \in \mathbb{Z}_p$.

By the Poly-LWR assumption with label set $\mathcal{L} \coloneqq \mathcal{X}$ the adversary cannot distinguish Game 1 from Game 2. In particular,

$$\left|\Pr\Big[\mathbf{G}_1(\lambda) = 1\Big] - \Pr\Big[\mathbf{G}_2(\lambda) = 1\Big]\right| \leq \epsilon_{\text{pLWR}}(\lambda) \tag{10}$$

for some negligible function ϵ_{pLWR}.

Game 3. In Game $\mathbf{G}_3(\lambda)$ we make a core change to Game $\mathbf{G}_2(\lambda)$: we change the way the challenger responds to a challenge query. At the beginning of the game the challenger samples a random function $f : \mathcal{X} \to \mathbb{Z}_p$ and responds to a query for $\mathsf{OChallenge}(x)$ with $f(x)$. We argue that for all $\mathcal{A}$

$$\left|\Pr\Big[\mathbf{G}_2(\lambda) = 1\Big] - \Pr\Big[\mathbf{G}_3(\lambda) = 1\Big]\right| \leq \Delta_{p,q}(\lambda) \cdot Q \tag{11}$$

where Q is the total number of evaluation and challenge queries from $\mathcal{A}$. To see why this holds, consider a particular $x \in \mathcal{X}$ for which $\mathcal{A}$ issued a challenge query. The fact that $\mathcal{A}$ could issue such a challenge query means that all the evaluation queries $\mathsf{OEval}(i, \mathcal{J}, x, \mathsf{ec})$ for x never requested t shares using a single ec. But then all the pre-rounded masks are independent uniform random values in $\mathbb{Z}_q$. After rounding to $\mathbb{Z}_p$ each mask $\widehat{mask}_i$ is statistically close to uniform in $\mathbb{Z}_p$ with negligible distance $\Delta_{p,q}$. In both games, therefore, the masks act as one-time pads for the evaluation shares. So the only difference between the two games is in the response to the challenge query for x. But there too, after rounding to $\mathbb{Z}_p$ the statistical distance between the games is the statistical distance between $f(x)$ and $\lfloor R_x(0) \rceil_p$ which is at most $\Delta_{p,q}$. Therefore the total statistical distance between the games is $\Delta_{p,q}(\lambda) \cdot Q$, as claimed in (11).

Game 4. In Game $\mathbf{G}_4(\lambda)$ we restore how the challenger responds to evaluation queries to the way it responded in Game $\mathbf{G}_0(\lambda)$. In particular, it responds to an evaluation query $\mathsf{OEval}(i, \mathcal{J}, x, \mathsf{ec})$ with $y_i + mask_i \in \mathbb{Z}_p$ where

$$y_i \leftarrow \big\lfloor \lambda_i \cdot \big\langle H(x), \mathbf{K}(i) \big\rangle \big\rceil_p \quad \text{and} \quad mask_i \leftarrow \big\lfloor \lambda_i \cdot \big\langle H'(x, \mathsf{ec}, \mathcal{J}), \mathbf{K}'(i) \big\rangle \big\rceil_p$$

By using the Poly-LWR assumption twice, as we did when we moved from $\mathbf{G}_0(\lambda)$ to $\mathbf{G}_2(\lambda)$, we obtain that

$$\left|\Pr\Big[\mathbf{G}_3(\lambda)=1\Big]-\Pr\Big[\mathbf{G}_4(\lambda)=1\Big]\right| \le \epsilon_{\mathrm{pLWR}}(\lambda)+\epsilon_{\mathrm{pLWR0}}(\lambda). \tag{12}$$

Completing the proof. The final game $\mathbf{G}_4(\lambda)$ is identical to the game $\mathbf{G}^{\mathrm{tprf}}_{\mathcal{A},F_{\mathrm{lat}}}(\lambda,1)$, and therefore

$$\Pr\Big[\mathbf{G}_4(\lambda)=1\Big]=\Pr\Big[\mathbf{G}^{\mathrm{tprf}}_{\mathcal{A},F_{\mathrm{lat}}}(\lambda,1)=1\Big]. \tag{13}$$

Putting (9)–(13) together, we obtain

$$\mathsf{Adv}^{\mathrm{tprf}}_{\mathcal{A},F_{\mathrm{lat}}}(\lambda)=\left|\Pr\Big[\mathbf{G}_4(\lambda)=1\Big]-\Pr\Big[\mathbf{G}_0(\lambda)=1\Big]\right| \le 2\epsilon_{\mathrm{pLWR}}(\lambda)+2\epsilon_{\mathrm{pLWR0}}(\lambda)+\Delta_{p,q}(\lambda)\cdot Q$$

which is a negligible function, as required. This completes the proof of Lemma 4.3. □

5 A hash-based context-aware threshold PRF

Our second context-aware threshold PRF is fully hash based, but is only efficient when $\binom{n}{t}$ is small. In particular, the size of every key share is $O_\lambda(\binom{n-1}{t-1})$.

Our starting point is a standard (non-threshold) secure PRF $F:\mathcal{K}\times\mathcal{X}\to\mathcal{Y}$ where $\mathcal{Y}=\{0,1\}^\kappa$ for some κ. To give some intuition, let us first use F to construct a 2-out-of-3 threshold PRF. The key generator samples three keys k_1,k_2,k_3 from $\mathcal{K}$ and sets $\mathsf{msk}:=(k_1,k_2,k_3)$. The threshold PRF is defined as

$$\hat{F}(\mathsf{msk},x):=F(k_1,x)\oplus F(k_2,x)\oplus F(k_3,x).$$

Next, the key generator gives

$$\{k_2,k_3\}\text{ to party 1},\qquad \{k_1,k_3\}\text{ to party 2},\qquad \text{and}\qquad \{k_1,k_2\}\text{ to party 3}.$$

It should be clear that every two parties can compute $\hat{F}(\mathsf{msk},x)$ for every $x\in\mathcal{X}$, but a single party cannot distinguish $\hat{F}(\mathsf{msk},x)$ from random.

We can generalize this to t-out-of-n, for general t and n, as follows. For now we describe a context-free scheme.

- $\mathsf{KeyGen}(1^\lambda,n,t)\to(\mathsf{msk},\mathsf{sk}_1,\ldots,\mathsf{sk}_n)$. The algorithm does:
  - set $s:=\binom{n}{t-1}$ and $u:=\binom{n-1}{t-1}$, (in the 2-of-3 example $s=3$ and $u=2$)
  - for every $S\in\binom{[n]}{t-1}$ sample k_S in $\mathcal{K}$, to get a total of s sampled keys,
  - set $\mathsf{msk}\leftarrow\left\{k_S\ :\ S\in\binom{[n]}{t-1}\right\}$ so that $|\mathsf{msk}|=s$,
  - for $i\in[n]$ set $\mathsf{sk}_i\leftarrow\left\{k_S\ :\ S\in\binom{[n]}{t-1}\ \wedge\ i\notin S\right\}$ so that $|\mathsf{sk}_i|=u$,
  - output $(\mathsf{msk},\mathsf{sk}_1,\ldots,\mathsf{sk}_n)$.
  - The PRF $\hat{F}:\mathcal{K}^s\times\mathcal{X}\to\mathcal{Y}$ is defined as

$$\hat{F}(\mathsf{msk},x):=\bigoplus_{S\in\binom{[n]}{t-1}}F(k_S,x). \tag{14}$$

- $\mathsf{Eval}(\mathsf{sk}_i,\ \mathcal{J} \in \binom{[n]}{t},\ x \in \mathcal{X}) \to y_i$. Define

$$\mathcal{L}_i := \left\{ S \in \binom{[n]}{t-1} \ :\ \min(\mathcal{J} \setminus S) = i \right\} \subset \binom{[n]}{t-1},$$

  where $\min(T)$ is the smallest element in the non-empty set $T \subseteq [n]$. Output

$$y_i \leftarrow \bigoplus_{S \in \mathcal{L}_i} F(k_S, x).$$

  Note that for all $S \in \binom{[n]}{t-1}$, since $|\mathcal{J}| = t$ and $|S| = t-1$, the set difference $\mathcal{J} \setminus S$ is not empty, so that $\min(\mathcal{J} \setminus S)$ is well defined. Moreover, for every $S \in \mathcal{L}_i$ we have $i \notin S$ and therefore sk_i contains the key k_S, as needed to compute y_i.

- $\mathsf{Combine}(\mathcal{J} \in \binom{[n]}{t},\ x,\ \{y_i\}_{i \in \mathcal{J}}) \to y$. Output $y \leftarrow \bigoplus_{i \in \mathcal{J}}\ y_i$.

Two key facts about this construction are:

- Every quorum $\mathcal{J}$ of t parties can compute $\hat{F}(\mathsf{msk}, x)$ from (14). To see why, observe that for every subset $S \in \binom{[n]}{t-1}$ the quantity $F(k_S, x)$ is xored into exactly one y_i, namely $i = \min(\mathcal{J} \setminus S)$. Therefore y is the xor of $F(k_S, x)$ for all $S \in \binom{[n]}{t-1}$ as required.

- A set of $t-1$ parties $S \in \binom{[n]}{t-1}$ cannot distinguish $\hat{F}(\mathsf{msk}, x)$ from random because no party in the set has the key k_S.

It is not difficult to see that for every combinatorial construction of this type, where msk is a tuple of independent PRF keys, the size of msk must be at least $\binom{n}{t-1}$. Therefore, this construction is optimal in this class of constructions. Some generalizations of this construction are explored in [34].

5.1 Making the scheme context aware

The remaining issue is a way to make this threshold PRF context-aware. In what follows we show how to compile the context-free threshold PRF above $(\mathsf{KeyGen}, \mathsf{Eval}, \mathsf{Combine})$ into a context-aware threshold PRF. To do so we will need a hash function

$$H : \mathcal{K} \times \mathcal{X} \times \mathcal{E} \times \{0,1\}^\lambda \to \mathcal{Y} \tag{15}$$

We construct a context-aware threshold PRF $\mathcal{F}_{\text{hash}} = (\mathsf{KeyGen}', \mathsf{Eval}', \mathsf{Combine}')$ as follows.

- $\mathsf{KeyGen}'(1^\lambda, n, t) \to (\mathsf{msk}, \mathsf{sk}'_1, \ldots, \mathsf{sk}'_n)$. The algorithm does:
  - run $\mathsf{KeyGen}(1^\lambda, n, t)$ to get $(\mathsf{msk}, \mathsf{sk}_1, \ldots, \mathsf{sk}_n)$
  - sample keys $k_{i,j}$ in $\mathcal{K}$ for all $i, j \in [n]$
  - for $i = 1, \ldots, n$ set $\mathsf{sk}'_i \leftarrow \left(\mathsf{sk}_i,\ \left\{k_{i,j}, k_{j,i}\ :\ j \in [n]\right\}\right)$
  - output $(\mathsf{msk}, \mathsf{sk}'_1, \ldots, \mathsf{sk}'_n)$
- $\mathsf{Eval}'(\mathsf{sk}'_i,\ \mathcal{J},\ x,\ \mathsf{ec}) \to y_i$ where $i \in \mathcal{J}$:

– run $y_i \leftarrow \mathsf{Eval}(\mathsf{sk}_i,\ \mathcal{J},\ x)$

– $mask_{i,1} \leftarrow \oplus_{j\in\mathcal{J}}\ H(k_{i,j}, x, \mathsf{ec}, \mathcal{J})$ and $mask_{i,2} \leftarrow \oplus_{j\in\mathcal{J}}\ H(k_{j,i}, x, \mathsf{ec}, \mathcal{J})$

– output $y'_i \leftarrow (y_i \ \oplus\ mask_{i,1}\ \oplus\ mask_{i,2})$

- $\mathsf{Combine}'(\mathcal{J} \in \binom{[n]}{t},\ x,\ \{y'_i\}_{i\in\mathcal{J}}) \to y$. Output $y \leftarrow \oplus_{i\in\mathcal{J}}\ y'_i$.

To see that the final y is equal to $\hat{F}(\mathsf{msk}, x)$ from (14), as required, observe that

$$\bigoplus_{i\in\mathcal{J}} (mask_{i,1} \oplus mask_{i,2}) = 0.$$

A similar masking technique is also used in the threshold Raccoon signature scheme [41]. These masks ensure that evaluation shares obtained under different evaluation contexts cannot be combined.

Theorem 5.1. *The threshold PRF $\mathcal{F}_{hash} = (\mathsf{KeyGen}', \mathsf{Eval}', \mathsf{Combine}')$ is adaptively secure (as in Definition 2.2) assuming F is a secure PRF, and the hash function H from (15) is modeled as a random oracle.*

Proof. The security of a threshold PRF in Definition 2.2 is defined using two games $\mathbf{G}^{\text{tprf}}_{\mathcal{A},\mathcal{F}_{\text{hash}}}(\lambda, b)$ for $b \in \{0,1\}$. We first modify these games to obtain the two games $\mathbf{G}^{\text{rtprf}}_{\mathcal{A},\mathcal{F}_{\text{hash}}}(\lambda, b)$ for $b \in \{0,1\}$ shown in Figure 5. In these games, the challenger first chooses a random set $T \in \binom{[n]}{t-1}$ and the game aborts if the adversary ever issues a corruption query that exposes the PRF key k_T to the adversary. The highlighted lines in Figure 5 show the differences from the games in Definition 2.2. The following lemma shows that it suffices to bound the adversary's advantage in distinguishing the two games in Figure 5.

Lemma 5.2. *For every adversary $\mathcal{A}$ that chooses parameters n and t we have*

$$\mathsf{Adv}^{\text{tprf}}_{\mathcal{A},\mathcal{F}_{hash}}(\lambda) \le \binom{n}{t-1} \cdot \left|\Pr\left[\mathbf{G}^{\text{rtprf}}_{\mathcal{A},\mathcal{F}_{hash}}(\lambda, 0) = 1\right] - \Pr\left[\mathbf{G}^{\text{rtprf}}_{\mathcal{A},\mathcal{F}_{hash}}(\lambda, 1) = 1\right]\right|. \tag{16}$$

The proof of Lemma 5.2 follows from two facts. First, the adversary cannot learn all the keys $\{k_S : S \in \binom{[n]}{t-1}\}$ by corrupting $t-1$ parties. Second, the chosen secret set T is independent of the adversary's view. In more detail, for $b \in \{0,1\}$ and using the random variable *fail* defined in Figure 5 we have

$$\begin{aligned}
\Pr\left[\mathbf{G}^{\text{rtprf}}_{\mathcal{A},\mathcal{F}_{\text{hash}}}(\lambda, b) = 1\right] &= \Pr\left[\mathbf{G}^{\text{tprf}}_{\mathcal{A},\mathcal{F}_{\text{hash}}}(\lambda, b) = 1 \ \wedge\ \neg \mathit{fail}\right] \\
&= \Pr\left[\mathbf{G}^{\text{tprf}}_{\mathcal{A},\mathcal{F}_{\text{hash}}}(\lambda, b) = 1\right] \cdot \Pr\left[\neg \mathit{fail}\right] \\
&= \Pr\left[\mathbf{G}^{\text{tprf}}_{\mathcal{A},\mathcal{F}_{\text{hash}}}(\lambda, b) = 1\right] / \binom{n}{t-1}
\end{aligned}$$

from which (16) follows. Note that line (11) in Figure 5 is there to ensure that $\Pr\left[\neg \mathit{fail}\right] = 1/\binom{n}{t-1}$. Without line (11) we could only conclude that $\Pr\left[\neg \mathit{fail}\right] \ge 1/\binom{n}{t-1}$, which is insufficient to imply (16). The inequality is strict if $\mathcal{A}$ corrupts fewer than $t-1$ parties. line (11) increases the probability of *fail* by artificially increasing the number of corrupted parties to $t-1$. This ensures that $\Pr\left[\neg \mathit{fail}\right]$ is exactly $1/\binom{n}{t-1}$.

We next bound the right hand side of (16) using a sequence of three hybrid games.

Game 1. The game $\mathbf{G}_1(\lambda)$ is identical to $\mathbf{G}^{\text{rtprf}}_{\mathcal{A},\mathcal{F}_{\text{hash}}}(\lambda, 0)$ except that we replace $F(k_T, \cdot)$ with a truly random function $g : \mathcal{X} \to \mathcal{Y}$. The resulting game is shown in Figure 6. When event *fail* happens then both games output 0 and are therefore indistinguishable. When event $\neg$*fail* happens, the key k_T is independent of the adversary's view. Consequently, because the PRF F is secure, we obtain

$$\Big|\Pr\big[\mathbf{G}^{\text{rtprf}}_{\mathcal{A},\mathcal{F}_{\text{hash}}}(\lambda, 0) = 1\big] - \Pr\big[\mathbf{G}_1(\lambda) = 1\big]\Big| = \Big|\Pr\big[\mathbf{G}^{\text{rtprf}}_{\mathcal{A},\mathcal{F}_{\text{hash}}}(\lambda, 0) = 1 \ \wedge \ \neg \mathit{fail}\big] - \Pr\big[\mathbf{G}_1(\lambda) = 1 \ \wedge \ \neg \mathit{fail}\big]\Big| \leq \epsilon_F(\lambda) \tag{17}$$

for some negligible function $\epsilon_F(\lambda)$.

Game 2. The game $\mathbf{G}_2(\lambda)$ is identical to $\mathbf{G}_1(\lambda)$ except that we replace the responses to challenge queries by truly random responses. Specifically, in Figure 6 we delete line (4) in OChallenge. We argue that

$$\Pr\big[\mathbf{G}_1(\lambda) = 1\big] = \Pr\big[\mathbf{G}_2(\lambda) = 1\big] \tag{18}$$

The Game $\mathbf{G}^{\text{rtprf}}_{\mathcal{A},\mathcal{F}_{\text{hash}}}(\lambda, \ b \in \{0,1\})$

1 : $C \leftarrow \emptyset$ // the set of challenge points
2 : $Q \leftarrow \emptyset$ // an associative array of queried points
3 : $D \leftarrow \emptyset$ // the set of corrupt parties
4 : $f \leftarrow\$ \ \text{Funs}[\mathcal{X}, \mathcal{Y}]$ // sample a random $f : \mathcal{X} \to \mathcal{Y}$
// sample the random oracle H
5 : $H \leftarrow\$ \ \text{Funs}[\mathcal{K} \times \mathcal{X} \times \mathcal{E} \times \{0,1\}^\lambda, \mathcal{Y}]$
6 : $(n, t, \mathsf{state}) \leftarrow\$ \ \mathcal{A}^H(1^\lambda)$
7 : $(\mathsf{msk}, \mathsf{sk}_1, \ldots, \mathsf{sk}_n) \leftarrow\$ \ \mathsf{KeyGen}(1^\lambda, n, t)$
// so $\mathsf{msk} = \{k_S \ : \ S \in \binom{[n]}{t-1}\}$
8 : $T \leftarrow\$ \ \binom{[n]}{t-1}$ // sample a random subset
9 : $\mathit{fail} \leftarrow \textbf{false}$
10 : $b' \leftarrow\$ \ \mathcal{A}^{\mathsf{OCorrupt},\mathsf{OEval},\mathsf{OChallenge},H}(1^\lambda, \mathsf{state})$
// the next line is an artificial abort needed for Lemma 5.2
11 : $D' \leftarrow\$ \ \binom{[n]\setminus D}{t-1-|D|}$; **if** $T \neq D \cup D'$: $\mathit{fail} \leftarrow \textbf{true}$
12 : **return** $b' \wedge (\neg \mathit{fail})$

Oracle $\mathsf{OChallenge}(x)$

1 : **if** $\exists \mathsf{ec} : \big|D \cup Q[x, \mathsf{ec}]\big| \geq t$:
2 : **return** $\perp$
3 : $y \leftarrow f(x)$
4 : **if** $b = 0 : y \leftarrow \hat{F}(\mathsf{msk}, x)$
5 : $C \leftarrow C \cup \{x\}$
6 : **return** y

Oracle $\mathsf{OEval}(i, \mathcal{J}, x, \mathsf{ec})$

1 : **if** $\big|D \cup Q[x, \mathsf{ec}] \cup \{i\}\big| \geq t$
2 : **and** $x \in C$: **return** $\perp$
3 : $Q[x, \mathsf{ec}] \leftarrow Q[x, \mathsf{ec}] \cup \{i\}$
4 : **return** $\mathsf{Eval}^H(1^\lambda, \mathsf{sk}_i, \mathcal{J}, x, \mathsf{ec})$

Oracle $\mathsf{OCorrupt}(i)$

1 : **if** $\exists \ x, \mathsf{ec}$ s.t. $\big|D \cup Q[x, \mathsf{ec}] \cup \{i\}\big| \geq t$
2 : **and** $x \in C$: **return** $\perp$
// fail if response reveals k_T to $\mathcal{A}$
3 : **if** $i \notin T : \mathit{fail} \leftarrow \textbf{true}$
4 : $D \leftarrow D \cup \{i\}$
5 : **return** sk_i

Figure 5: Games $\mathbf{G}^{\text{rtprf}}_{\mathcal{A},\mathcal{F}_{\text{hash}}}(\lambda, b)$ in the proof of Theorem 5.1 for $b \in \{0,1\}$. The highlighted lines mark the changes from games $\mathbf{G}^{\text{tprf}}_{\mathcal{A},\mathcal{F}_{\text{hash}}}(\lambda, b)$ in Definition 2.2.

First, observe that for every challenge point $x \in \mathcal{X}$, if $g(x)$ is independent of the adversary's view, then the response to an $\mathsf{OChallenge}(x)$ query in both $\mathbf{G}_1(\lambda)$ and $\mathbf{G}_2(\lambda)$ is distributed as an independent uniform sample in $\mathcal{Y}$. Second, for a challenge point x we know that for every ec and $\mathcal{J}$, the adversary made queries to $\mathsf{OEval}(i, \mathcal{J}, x, \mathsf{ec})$ for at most $t-1-|D|$ distinct $i \in \mathcal{J}$, where D is the final set of corrupt parties. Consider one such query $\mathsf{OEval}(i, \mathcal{J}, x, \mathsf{ec})$. Since $|\mathcal{J} \setminus D| \geq t - |D|$, there must be some $j \in \mathcal{J} \setminus D$ so that the adversary never queried for $\mathsf{OEval}(j, \mathcal{J}, x, \mathsf{ec})$. But then $H(k_{i,j}, x, \mathsf{ec}, \mathcal{J})$ is used in $mask_{i,1}$ and in no other query to OEval. As a result, the response y'_i to $\mathsf{OEval}(i, \mathcal{J}, x, \mathsf{ec})$ is a fresh uniform value in $\mathcal{Y}$. Hence, the responses to all queries $\mathsf{OEval}(\cdot, \cdot, x, \cdot)$, for a challenge point x, are independent uniform random values in $\mathcal{Y}$. Therefore $g(x)$ is independent of the adversary's view, which implies (18).

Game 3. The game $\mathbf{G}_3(\lambda)$ is identical to $\mathbf{G}_2(\lambda)$ except that we make the responses to evaluation queries use the correct PRF value $F(k_T, x)$. Specifically, in Figure 6 we replace line (7) in OEval with $y_i \leftarrow y_i \oplus F(k_T, x)$. This only changes the adversary's view for non-challenge points $x \in \mathcal{X}$, namely points where it queried $\mathsf{OEval}(i, \mathcal{J}, x, \mathsf{ec})$ at t or more values of i. Because the PRF F is

The Game $\mathbf{G}_1(\lambda)$

1 : $C \leftarrow \emptyset$ // the set of challenge points
2 : $Q \leftarrow \emptyset$ // an associative array of queried points
3 : $D \leftarrow \emptyset$ // the set of corrupt parties
4 : $g \leftarrow\$ \mathrm{Funs}[\mathcal{X}, \mathcal{Y}]$ // we use $g(x)$ instead of $F(k_T, x)$
// sample the random oracle H
5 : $H \leftarrow\$ \mathrm{Funs}[\mathcal{K} \times \mathcal{X} \times \mathcal{E} \times \{0,1\}^\lambda, \mathcal{Y}]$
6 : $(n, t, \mathsf{state}) \leftarrow\$ \mathcal{A}^H(1^\lambda)$
7 : $(\mathsf{msk}, \mathsf{sk}_1, \ldots, \mathsf{sk}_n) \leftarrow\$ \mathsf{KeyGen}(1^\lambda, n, t)$
// So $\mathsf{msk} = \{k_S \ : \ S \in \binom{[n]}{t-1}\}$
8 : $T \leftarrow\$ \binom{[n]}{t-1}$ // sample a random subset
9 : $fail \leftarrow \mathbf{false}$
10 : $b' \leftarrow\$ \mathcal{A}^{\mathsf{OCorrupt}, \mathsf{OEval}, \mathsf{OChallenge}, H}(1^\lambda, \mathsf{state})$
// the next line is an artificial abort needed for Lemma 5.2
11 : $D' \leftarrow\$ \binom{[n] \setminus D}{t-1-|D|}$; **if** $T \neq D \cup D'$: $fail \leftarrow \mathbf{true}$
12 : **return** $b' \wedge (\neg fail)$

Oracle $\mathsf{OCorrupt}(i)$

1 : **if** $\exists\, x, \mathsf{ec}$ s.t. $\left|D \cup Q[x, \mathsf{ec}] \cup \{i\}\right| \geq t$
2 : **and** $x \in C$: **return** $\perp$
3 : **if** $i \notin T$: $fail \leftarrow \mathbf{true}$ // response reveals k_T to $\mathcal{A}$
4 : $D \leftarrow D \cup \{i\}$
5 : **return** sk_i

Oracle $\mathsf{OChallenge}(x)$

1 : **if** $\exists \mathsf{ec} : \left|D \cup Q[x, \mathsf{ec}]\right| \geq t$:
2 : **return** $\perp$
3 : $y \leftarrow g(x)$ // random value for $F(K_T, x)$
4 : $y \leftarrow y \oplus \bigoplus_{S \neq T} F(k_S, x)$
5 : $C \leftarrow C \cup \{x\}$
6 : **return** y

Oracle $\mathsf{OEval}(i, \mathcal{J}, x, \mathsf{ec})$

1 : **if** $\left|D \cup Q[x, \mathsf{ec}] \cup \{i\}\right| \geq t$
2 : **and** $x \in C$: **return** $\perp$
3 : $Q[x, \mathsf{ec}] \leftarrow Q[x, \mathsf{ec}] \cup \{i\}$
// compute $y'_i \leftarrow \mathsf{Eval}(1^\lambda, \mathsf{sk}_i, \mathcal{J}, x, \mathsf{ec})$ using $g(x)$ for $F(K_T, x)$
4 : $\mathcal{L}_i \leftarrow \left\{S \in \binom{[n]}{t-1} \ : \ \min(\mathcal{J} \setminus S) = i\right\}$
5 : $y_i \leftarrow \bigoplus_{T \neq S \in \mathcal{L}_i} F(k_S, x)$ // exclude $F(K_T, x)$
6 : **if** $T \in \mathcal{L}_i$: // if $F(K_T, x)$ is needed,
7 : $y_i \leftarrow y_i \oplus g(x)$ // use $g(x)$ instead
8 : $mask_{i,1} \leftarrow \oplus_{j \in \mathcal{J}} \ H(k_{i,j}, x, \mathsf{ec}, \mathcal{J})$
9 : $mask_{i,2} \leftarrow \oplus_{j \in \mathcal{J}} \ H(k_{j,i}, x, \mathsf{ec}, \mathcal{J})$
10 : $y'_i \leftarrow y_i \oplus mask_{i,1} \oplus mask_{i,2}$
11 : **return** y'_i

Figure 6: The game $\mathbf{G}_1(\lambda)$ in the proof of Theorem 5.1. The shaded lines are modified to obtain games $\mathbf{G}_2(\lambda)$ and $\mathbf{G}_3(\lambda)$.

secure, we obtain

$$\Big|\Pr\big[\mathbf{G}_2(\lambda)=1\big]-\Pr\big[\mathbf{G}_3(\lambda)=1\big]\Big|\leq\epsilon_F \tag{19}$$

Completing the proof. It is not difficult to see that the final game $\mathbf{G}_3(\lambda)$ is identical to the game $\mathbf{G}^{\mathrm{rtprf}}_{\mathcal{A},\mathcal{F}_{\mathrm{hash}}}(\lambda,1)$ from Figure 5, and therefore

$$\Pr\big[\mathbf{G}_3(\lambda)=1\big]=\Pr\big[\mathbf{G}^{\mathrm{rtprf}}_{\mathcal{A},\mathcal{F}_{\mathrm{hash}}}(\lambda,1)=1\big] \tag{20}$$

Putting (16) – (20) together we obtain that

$$\mathsf{Adv}^{\mathrm{tprf}}_{\mathcal{A},\mathcal{F}_{\mathrm{hash}}}(\lambda)\leq 2\tbinom{n}{t-1}\cdot\epsilon_F(\lambda)$$

which is negligible since $\epsilon_F(\lambda)$ is negligible in λ, but $\binom{n}{t-1}$ is polynomial in λ. □

6 Extensions and open problems

This work raises a number of directions for future work.

Updating n and t and proactive refresh. A question that comes up often in practice is how to change the set of signing parties without changing the public key. This question can be recast as a question about changing n or t without changing the public key. In our setting we can further ask how to change n or t without changing the combiner's public key cpk. A related question, called proactive refresh, is how to refresh the key shares without changing any of $\mathsf{pk}, \mathsf{cpk}, n, t$. Since our context-aware lattice threshold PRF is based on Shamir secret sharing all of these operations can be done using standard update protocols in the literature.

Adaptive security. While our hash-based context-aware threshold PRF is adaptively secure, we only prove static security of the lattice threshold PRF. Libert, Stehlé, and Titiu [33] designed an adaptively-secure lattice threshold PRF. It would be interesting to extend that construction and make it context-aware, possibly using a similar masking technique as in Section 4.

Distributed key generation (DKG). A DKG protocol for our scheme would require evaluating the one-way function $f : \mathcal{Y} \to \mathcal{Y}$ in an MPC to generate the combiner's public key cpk. This can be costly, although recent work [35] suggests that for small n it is not impractical thanks to recent advances in MPC. An interesting direction for future work is to design an efficient DKG protocol for hash-based signatures that is practical for modest values of n and t, say for a 20-out-of-30 scheme.

Silent setup. Recent work on threshold signing with a silent setup [24, 30, 47, 23] enables the parties to dynamically choose n and t for every signature without any interaction among the parties. Our framework is not designed for a silent setup. It would be quite interesting if there were a way to extend this framework to support a silent setup.

Acknowledgments

We thank Conduition for suggesting hypertree pruning as a way to apply our approach to the Shrincs signature scheme. This work was partially supported by the Stanford Center for Blockchain Research (CBR) and by UBRI. In addition, Bünz was partially supported by Chaincode.

References

[1] Mehdi Abri and Jonathan Katz. *Shorter Hash-Based Signatures Using Forced Pruning*. Cryptology ePrint Archive, Report 2025/2069. 2025. URL: https://eprint.iacr.org/2025/2069.

[2] Alexandre Adomnicăi. *Towards threshold hash-based signatures for post-quantum distributed validators*. URL: https://github.com/ObolNetwork/pqdv/blob/main/doc/main.pdf.

[3] Jean-Philippe Aumasson and Guillaume Endignoux. *Improving Stateless Hash-Based Signatures*. Cryptology ePrint Archive, Report 2017/933. 2017. URL: https://eprint.iacr.org/2017/933.

[4] Abhishek Banerjee and Chris Peikert. "New and Improved Key-Homomorphic Pseudorandom Functions". In: *CRYPTO 2014, Part I*. Ed. by Juan A. Garay and Rosario Gennaro. Vol. 8616. LNCS. Springer, Berlin, Heidelberg, Aug. 2014, pp. 353–370. DOI: 10.1007/978-3-662-44371-2_20.

[5] Abhishek Banerjee, Chris Peikert, and Alon Rosen. "Pseudorandom Functions and Lattices". In: *EUROCRYPT 2012*. Ed. by David Pointcheval and Thomas Johansson. Vol. 7237. LNCS. Springer, Berlin, Heidelberg, Apr. 2012, pp. 719–737. DOI: 10.1007/978-3-642-29011-4_42.

[6] Daniel J. Bernstein, Andreas Hülsing, Stefan Kölbl, Ruben Niederhagen, Joost Rijneveld, and Peter Schwabe. "The SPHINCS$^+$ Signature Framework". In: *ACM CCS 2019*. Ed. by Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz. ACM Press, Nov. 2019, pp. 2129–2146. DOI: 10.1145/3319535.3363229.

[7] Dan Boneh, Benedikt Bünz, Kartik Nayak, Lior Rotem, and Victor Shoup. "Context-Dependent Threshold Decryption and Its Applications". In: *ASIACRYPT 2025, Part VI*. Ed. by Goichiro Hanaoka and Bo-Yin Yang. Vol. 16250. LNCS. Springer, Singapore, Dec. 2025, pp. 506–538. DOI: 10.1007/978-981-95-5119-4_16.

[8] Dan Boneh, Rosario Gennaro, Steven Goldfeder, Aayush Jain, Sam Kim, Peter M. R. Rasmussen, and Amit Sahai. "Threshold Cryptosystems from Threshold Fully Homomorphic Encryption". In: *CRYPTO 2018, Part I*. Ed. by Hovav Shacham and Alexandra Boldyreva. Vol. 10991. LNCS. Springer, Cham, Aug. 2018, pp. 565–596. DOI: 10.1007/978-3-319-96884-1_19.

[9] Dan Boneh, Kevin Lewi, Hart William Montgomery, and Ananth Raghunathan. "Key Homomorphic PRFs and Their Applications". In: *CRYPTO 2013, Part I*. Ed. by Ran Canetti and Juan A. Garay. Vol. 8042. LNCS. Springer, Berlin, Heidelberg, Aug. 2013, pp. 410–428. DOI: 10.1007/978-3-642-40041-4_23.

[10] Dan Boneh and Victor Shoup. *A Graduate Course in Applied Cryptography (version 0.6)*. Available at https://toc.cryptobook.us/. 2023.

[11] Zvika Brakerski, Offir Friedman, Avichai Marmor, Dolev Mutzari, Yuval Spiizer, and Ni Trieu. *Threshold FHE with Efficient Asynchronous Decryption*. Cryptology ePrint Archive, Report 2025/712. 2025. URL: https://eprint.iacr.org/2025/712.

[12] Johannes Buchmann, Erik Dahmen, and Andreas Hülsing. *XMSS - A Practical Forward Secure Signature Scheme based on Minimal Security Assumptions*. Cryptology ePrint Archive, Report 2011/484. 2011. URL: https://eprint.iacr.org/2011/484.

[13] Christian Cachin, Klaus Kursawe, and Victor Shoup. *Random Oracles in Constantinople: Practical Asynchronous Byzantine Agreement using Cryptography*. Cryptology ePrint Archive, Report 2000/034. 2000. URL: https://eprint.iacr.org/2000/034.

[14] Conduition. *Pruning Hypertrees for the Lax and Lazy*. 2026. URL: conduition.io/cryptography/hypertree-pruning.

[15] Conduition, Ethan Heilman, Mikhail Kudinov, Oleksandr Kurbatov, Jonas Nick, and remix7531. *SHRINCS: A Compact Hash-Based Signature Scheme*. https://github.com/SHRINCS/shrincs-bip/blob/main/SHRINCS.md. 2026.

[16] Daniele Cozzo and Nigel P. Smart. "Sharing the LUOV: Threshold Post-quantum Signatures". In: *17th IMA International Conference on Cryptography and Coding*. Ed. by Martin Albrecht. Vol. 11929. LNCS. Springer, Cham, Dec. 2019, pp. 128–153. DOI: 10.1007/978-3-030-35199-1_7.

[17] Ivan Damgård and Rune Thorbek. "Linear Integer Secret Sharing and Distributed Exponentiation". In: *PKC 2006*. Ed. by Moti Yung, Yevgeniy Dodis, Aggelos Kiayias, and Tal Malkin. Vol. 3958. LNCS. Springer, Berlin, Heidelberg, Apr. 2006, pp. 75–90. DOI: 10.1007/11745853_6.

[18] Quynh Dang and Dustin Moody. *Additional SLH-DSA Parameter Sets for Limited-Signature Use Cases*. NIST SP 800-230 ipd. 2026. DOI: https://doi.org/10.6028/NIST.SP.800-230.ipd.

[19] Yevgeniy Dodis. "Efficient Construction of (Distributed) Verifiable Random Functions". In: *PKC 2003*. Ed. by Yvo Desmedt. Vol. 2567. LNCS. Springer, Berlin, Heidelberg, Jan. 2003, pp. 1–17. DOI: 10.1007/3-540-36288-6_1.

[20] Yevgeniy Dodis, Aleksandr Yampolskiy, and Moti Yung. "Threshold and Proactive Pseudo-Random Permutations". In: *TCC 2006*. Ed. by Shai Halevi and Tal Rabin. Vol. 3876. LNCS. Springer, Berlin, Heidelberg, Mar. 2006, pp. 542–560. DOI: 10.1007/11681878_28.

[21] Justin Drake, Dmitry Khovratovich, Mikhail Kudinov, and Benedikt Wagner. *Technical Note: LeanSig for Post-Quantum Ethereum*. Cryptology ePrint Archive, Report 2025/1332. 2025. URL: https://eprint.iacr.org/2025/1332.

[22] Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Prest, Thomas Ricosset, Gregor Seiler, William Whyte, and Zhenfei Zhang. *Falcon: Fast-Fourier Lattice-based Compact Signatures over NTRU*. Official Specification and NIST Submission. Available at falcon-sign.info/falcon.pdf. 2020.

[23] Preshtha Garg, Sanjam Garg, Guru-Vamsi Policharla, and Bhaskar Roberts. *Threshold Anonymous Credentials with Silent Setup*. Cryptology ePrint Archive, Report 2025/2042. 2025. URL: https://eprint.iacr.org/2025/2042.

[24] Sanjam Garg, Abhishek Jain, Pratyay Mukherjee, Rohit Sinha, Mingyuan Wang, and Yinuo Zhang. "hinTS: Threshold Signatures with Silent Setup". In: *2024 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 2024, pp. 3034–3052. DOI: 10.1109/SP54263.2024.00057.

[25] Sergey Gorbunov, Vinod Vaikuntanathan, and Daniel Wichs. "Leveled Fully Homomorphic Signatures from Standard Lattices". In: *47th ACM STOC*. Ed. by Rocco A. Servedio and Ronitt Rubinfeld. ACM Press, June 2015, pp. 469–477. DOI: 10.1145/2746539.2746576.

[26] Shai Halevi and Hugo Krawczyk. "Strengthening Digital Signatures Via Randomized Hashing". In: *CRYPTO 2006*. Ed. by Cynthia Dwork. Vol. 4117. LNCS. Springer, Berlin, Heidelberg, Aug. 2006, pp. 41–59. DOI: 10.1007/11818175_3.

[27] Shlomo Hoory, Avner Magen, and Toniann Pitassi. "Monotone Circuits for the Majority Function". In: *APPROX-RANDOM 2006*. Vol. 4110. Springer, 2006, pp. 410–425. DOI: 10.1007/11830924_38.

[28] Andreas Hülsing. *WOTS+ – Shorter Signatures for Hash-Based Signature Schemes*. Cryptology ePrint Archive, Report 2017/965. 2017. URL: https://eprint.iacr.org/2017/965.

[29] Andreas Hülsing, Joost Rijneveld, and Fang Song. *Mitigating Multi-Target Attacks in Hash-based Signatures*. Cryptology ePrint Archive, Report 2015/1256. 2015. URL: https://eprint.iacr.org/2015/1256.

[30] Aniket Kate, Pratyay Mukherjee, Samipa Samanta, and Pratik Sarkar. *Dyna-hinTS: Silent Threshold Signatures for Dynamic Committees*. Cryptology ePrint Archive, Report 2025/631. 2025. URL: https://eprint.iacr.org/2025/631.

[31] John Kelsey, Nathalie Lang, and Stefan Lucks. "Turning Hash-Based Signatures into Distributed Signatures and Threshold Signatures: Delegate Your Signing Capability, and Distribute it Among Trustees". In: *CiC* 2.2 (2025), p. 24. DOI: 10.62056/a6ksudy6b.

[32] Mikhail Kudinov and Jonas Nick. *Hash-based Signature Schemes for Bitcoin*. Cryptology ePrint Archive, Report 2025/2203. 2025. URL: https://eprint.iacr.org/2025/2203.

[33] Benoît Libert, Damien Stehlé, and Radu Titiu. "Adaptively Secure Distributed PRFs from LWE". In: *TCC 2018, Part II*. Ed. by Amos Beimel and Stefan Dziembowski. Vol. 11240. LNCS. Springer, Cham, Nov. 2018, pp. 391–421. DOI: 10.1007/978-3-030-03810-6_15.

[34] Silvio Micali and Ray Sidney. "A Simple Method for Generating and Sharing Pseudo-Random Functions, with Applications to Clipper-like Escrow Systems". In: *CRYPTO'95*. Ed. by Don Coppersmith. Vol. 963. LNCS. Springer, Berlin, Heidelberg, Aug. 1995, pp. 185–196. DOI: 10.1007/3-540-44750-4_15.

[35] Isabel Muñoz, Isaac Agudo, Marco López, and Daniel Morales. *Dealing Haystack: Towards Trustless Haystack in the Optimistic Setting*. Cryptology ePrint Archive, Paper 2026/1804. 2026. URL: https://eprint.iacr.org/2026/1804.

[36] Moni Naor, Benny Pinkas, and Omer Reingold. "Distributed Pseudo-random Functions and KDCs". In: *EUROCRYPT'99*. Ed. by Jacques Stern. Vol. 1592. LNCS. Springer, Berlin, Heidelberg, May 1999, pp. 327–346. DOI: 10.1007/3-540-48910-X_23.

[37] Moni Naor and Omer Reingold. "Number-theoretic Constructions of Efficient Pseudo-random Functions". In: *38th FOCS*. IEEE Computer Society Press, Oct. 1997, pp. 458–467. DOI: 10.1109/SFCS.1997.646134.

[38] Jesper Buus Nielsen. "A Threshold Pseudorandom Function Construction and Its Applications". In: *CRYPTO 2002*. Ed. by Moti Yung. Vol. 2442. LNCS. Springer, Berlin, Heidelberg, Aug. 2002, pp. 401–416. DOI: 10.1007/3-540-45708-9_26.

[39] NIST. *Module-Lattice-Based Digital Signature Standard*. NIST FIPS 204. 2024. DOI: 10.6028/NIST.FIPS.204.

[40] NIST. *Stateless Hash-Based Digital Signature Standard*. NIST FIPS 205. 2024. DOI: 10.6028/NIST.FIPS.205.

[41] Rafaël del Pino, Shuichi Katsumata, Mary Maller, Fabrice Mouhartem, Thomas Prest, and Markku-Juhani Saarinen. *Threshold Raccoon: Practical Threshold Signatures from Standard Lattice Assumptions*. Cryptology ePrint Archive, Report 2024/184. 2024. URL: https://eprint.iacr.org/2024/184.

[42] Oded Regev. "On lattices, learning with errors, random linear codes, and cryptography". In: *J. ACM* 56.6 (2009), 34:1–34:40. DOI: 10.1145/1568318.1568324.

[43] Leonid Reyzin and Natan Reyzin. "Better than BiBa: Short One-Time Signatures with Fast Signing and Verifying". In: *ACISP 02*. Ed. by Lynn Margaret Batten and Jennifer Seberry. Vol. 2384. LNCS. Springer, Berlin, Heidelberg, July 2002, pp. 144–153. DOI: 10.1007/3-540-45450-0_11.

[44] Sayani Sinha, Sikhar Patranabis, and Debdeep Mukhopadhyay. *Efficient Quantum-safe Distributed PRF and Applications: Playing DiSE in a Quantum World*. Cryptology ePrint Archive, Report 2025/152. 2025. URL: https://eprint.iacr.org/2025/152.

[45] Srivatsan Sridhar, Alberto Sonnino, and Lefteris Kokoris-Kogias. *Stingray: Fast Concurrent Transactions Without Consensus*. 2025. arXiv: 2501.06531. URL: https://arxiv.org/abs/2501.06531.

[46] Leslie G. Valiant. "Short Monotone Formulae for the Majority Function". In: *J. Algorithms* 5.3 (1984), pp. 363–366. DOI: 10.1016/0196-6774(84)90016-6.

[47] Brent Waters and David J. Wu. "Silent Threshold Cryptography from Pairings: Expressive Policies in the Plain Model". In: *EUROCRYPT 2026, Part V*. Ed. by Joan Daemen and Emmanuel Thomé. Vol. 16545. LNCS. Springer, Cham, May 2026, pp. 517–547. DOI: 10.1007/978-3-032-25330-9_18.

A Proof of Lemma 4.2

Lemma 4.2. *The Poly-LWR assumption holds with parameters p, q, d, N assuming that LWE is hard with parameters q, χ, d, m where $m = \mathsf{poly}(\lambda)$, and $B_\chi N^{4N}(p/q)$ is a negligible function.*

Proof. Let $\mathcal{A}$ be a Poly-LWR adversary that distinguishes the two Poly-LWR games in Figure 4. We construct an adversary $\mathcal{B}$ that distinguishes the two LWE distributions in Definition 4.1 with about the same advantage and running time as $\mathcal{A}$. This $\mathcal{B}$ works as follows.

Multi-LWE Challenge: $\mathcal{B}$ runs $\mathcal{A}(1^\lambda)$ and obtains $(n, t, D, \mathsf{state})$, where $|D| < t$ and $t \le n \le N$. Let $w := t - |D|$. Our $\mathcal{B}$ will use $\mathcal{A}$ to break a given w-multi-LWE challenge (LWE with w secret vectors $\mathbf{s}_1, \ldots, \mathbf{s}_w$). That is, $\mathcal{B}$ takes as input $(\mathbf{A}, \mathbf{Z})$, where $\mathbf{A} \in \mathbb{Z}_q^{m \times d}$ and $\mathbf{Z} \in \mathbb{Z}_q^{m \times w}$. Its goal is to decide if

$$\big(\mathbf{Z} = \mathbf{AS} + \mathbf{E} \ \text{ where } \mathbf{S} \in \mathbb{Z}_q^{d \times w} \text{ and } \mathbf{E} \leftarrow\!\$\ \chi^{m \times w}\big) \quad \text{or} \quad \big(\mathbf{Z} \leftarrow\!\$\ \mathbb{Z}_q^{m \times w}\big).$$

Let $\mathbf{a}_1, \ldots, \mathbf{a}_m \in \mathbb{Z}_q^d$ be the rows of $\mathbf{A}$, and let $\mathbf{s}_1, \ldots, \mathbf{s}_w \in \mathbb{Z}_q^d$ be the columns of $\mathbf{S}$. Finally, we assume that m is more than the number of Opoly queries that $\mathcal{A}$ will make when it runs.

Init: For every $j \in D$ our $\mathcal{B}$ samples $\mathbf{r}_j \leftarrow\!\$\ \mathbb{Z}_q^d$. Choose arbitrary $d_1, \ldots, d_w \in [n] \setminus D$ and set $D' := D \cup \{d_1, \ldots, d_w\}$. Then $|D'| = t$.

Running $\mathcal{A}$: $\mathcal{B}$ initializes an empty associative array T and a counter $c \leftarrow 1$. Next, it runs $\mathcal{A}^{\mathsf{Opoly},\mathsf{OH}}\Big(1^\lambda, \{\mathbf{r}_j\}_{j \in D}, \mathsf{state}\Big)$.

- When $\mathcal{A}$ issues a query for $\mathsf{OH}(\ell)$ for $\ell \in \mathcal{L}$, our $\mathcal{B}$ responds as:
  - if $\hat{\mathbf{a}}_\ell$ is undefined, set

$$T[\ell] \leftarrow c, \qquad \hat{\mathbf{a}}_\ell \leftarrow (n!)^4 \cdot \mathbf{a}_c \in \mathbb{Z}_q^d, \qquad c \leftarrow c + 1$$

  - return $\hat{\mathbf{a}}_\ell \in \mathbb{Z}_q^d$

  By assumption, m in the LWE challenge is sufficiently large so that c never exceeds m. The $(n!)^4$ term absorbed into $\hat{\mathbf{a}}_\ell$ will be used to cancel the denominators in the Lagrange coefficients below.
- When $\mathcal{A}$ issues a query for $\mathsf{Opoly}(i, \mathcal{J}, \ell)$, we can assume without loss of generality that $\hat{\mathbf{a}}_\ell$ is already defined. Then $\mathcal{B}$ responds with

$$\left\lfloor \lambda_i^{\mathcal{J}} \sum_{u \in D} \lambda_u^{D'}(i) \cdot \big\langle \hat{\mathbf{a}}_\ell, \mathbf{r}_u \big\rangle + \sum_{v=1}^{w} \big((n!)^4 \lambda_i^{\mathcal{J}} \lambda_{d_v}^{D'}(i)\big) \cdot \mathbf{Z}[T[\ell], v] \right\rceil_p$$

  where for $u \in D'$ we denote by $\lambda_u^{D'}(i) \in \mathbb{Z}_q$ the Lagrange coefficient for u when interpolating the value of a polynomial at the point $X = i$ using its values at the t points in D'. Note that $\lambda_j^{\mathcal{J}} = \lambda_j^{\mathcal{J}}(0)$.

Output: Finally, $\mathcal{B}$ returns whatever $\mathcal{A}$ returns.

This completes the description of $\mathcal{B}$. It remains to analyze its advantage in breaking w-multi-LWE.

LWE case. Suppose $\mathbf{Z} = \mathbf{AS} + \mathbf{E}$. Let $\mathbf{K} \in (\mathbb{Z}_q^{(<t)}[X])^d$ be the unique vector of polynomials that satisfies

$$\mathbf{K}(u) = \mathbf{r}_u \text{ for } u \in D \quad \text{and} \quad \mathbf{K}(d_v) = \mathbf{s}_v \text{ for } v \in [w]$$

Then, $\mathcal{B}$'s response to a query for $\mathsf{Opoly}(i, \mathcal{J}, \ell)$ can be written as

$$\left\lfloor \lambda_i^{\mathcal{J}} \left(\sum_{u \in D} \lambda_u^{D'}(i) \langle \hat{\mathbf{a}}_\ell, \mathbf{r}_u) \rangle + \sum_{v=1}^{w} \lambda_{d_v}^{D'}(i) (\langle \hat{\mathbf{a}}_\ell, \mathbf{s}_v \rangle + (n!)^4 \mathbf{E}[T[\ell], v]) \right) \right\rceil_p =$$

$$\left\lfloor \lambda_i^{\mathcal{J}} \langle \hat{\mathbf{a}}_\ell, \mathbf{K}(i) \rangle + \sum_{v=1}^{w} (n!)^4 \lambda_i^{\mathcal{J}} \lambda_{d_v}^{D'}(i) \cdot \mathbf{E}[T[\ell], v] \right\rceil_p = \left\lfloor \lambda_i^{\mathcal{J}} \langle \hat{\mathbf{a}}_\ell, \mathbf{K}(i) \rangle + \Delta \right\rceil_p$$

where

$$|\Delta| \leq \sum_{v=1}^{w} \left| (n!)^4 \lambda_i^{\mathcal{J}} \lambda_{d_v}^{D'}(i) \cdot B_\chi \right| = B_\chi \cdot \sum_{v=1}^{w} \left| (n!)^4 \lambda_i^{\mathcal{J}} \lambda_{d_v}^{D'}(i) \right| \leq B_\chi \cdot (w n^{4n})$$

The last inequality follows from the fact that, for every $v \in [w]$, the coefficient $(n!)^4 \lambda_i^{\mathcal{J}} \lambda_{d_v}^{D'}(i) \in \mathbb{Z}_q$, lifted to the integers in $[-q, q]$, lies in the interval $[-n^{4n}, n^{4n}]$. We see that $\mathcal{B}$'s response is equal to $\lfloor \lambda_i^{\mathcal{J}} \langle \hat{\mathbf{a}}_\ell, \mathbf{K}(i) \rangle \rceil_p$, as in $\mathbf{G}^{\text{pLWR}}_{\mathcal{A},(p,q,d,N)}(\lambda, 0)$, except with probability at most $\epsilon_{\text{rnd}}(\lambda) \coloneqq B_\chi \cdot w n^{4n} \cdot (p/q)$. By assumption $\epsilon_{\text{rnd}}(\lambda)$ is a negligible function.

Uniform case. Suppose $\mathbf{Z} \leftarrow\!\$\, \mathbb{Z}_q^{m \times w}$. For each $\ell \in \mathcal{L}$ where $\mathcal{A}$ queried for $\mathsf{OH}(\ell)$, let $R_\ell \in \mathbb{Z}_q^{(<t)}[X]$ be the unique polynomial that satisfies

$$R_\ell(u) = \langle \hat{\mathbf{a}}_\ell, \mathbf{r}_u \rangle \text{ for } u \in D \quad \text{and} \quad R_\ell(d_v) = (n!)^4 \, \mathbf{Z}[T[\ell], v] \text{ for } v = 1, \ldots, w$$

Then, $\mathcal{B}$'s response to a query for $\mathsf{Opoly}(i, \mathcal{J}, \ell)$ is

$$\left\lfloor \lambda_i^{\mathcal{J}} \left(\sum_{u \in D} \lambda_u^{D'}(i) \langle \hat{\mathbf{a}}_\ell, \mathbf{r}_u) \rangle + \sum_{v=1}^{w} \lambda_{d_v}^{D'}(i) \cdot (n!)^4 \, \mathbf{Z}[T[\ell], v] \right) \right\rceil_p = \left\lfloor \lambda_i^{\mathcal{J}} \cdot R_\ell(i) \right\rceil_p$$

which is exactly the response in $\mathbf{G}^{\text{pLWR}}_{\mathcal{A},(p,q,d,N)}(\lambda, 1)$.

Completing the proof. We showed that when $\mathcal{B}$ receives valid LWE samples, it simulates $\mathbf{G}^{\text{pLWR}}_{\mathcal{A},(p,q,d,N)}(\lambda, 0)$, up to an error $Q\epsilon_{\text{rnd}}(\lambda)$, where Q is the number of queries to Opoly. When the samples are uniform, $\mathcal{B}$ perfectly simulates $\mathbf{G}^{\text{pLWR}}_{\mathcal{A},(p,q,d,N)}(\lambda, 1)$. Therefore,

$$\mathsf{Adv}^{\text{plwr}}_{\mathcal{A},(p,q,d,N)}(\lambda) \leq w \cdot \mathsf{Adv}^{\text{lwe}}_{\mathcal{B},q,\chi,d,m}(\lambda) + Q\epsilon_{\text{rnd}}(\lambda), \tag{21}$$

Hence, if LWE holds for parameters q, χ, d, m and $\epsilon_{\text{rnd}}(\lambda) = B_\chi w n^{4n}(p/q)$ is negligible, then $\mathcal{A}$'s advantage must also be negligible, as required. The proof that $\mathsf{Adv}^{\text{plwr0}}_{\mathcal{A},(p,q,d,N)}(\lambda)$ from Definition 4.4 is also negligible is essentially the same, by further requiring that the sampled polynomials evaluate to zero at $X = 0$. □